Received: 31 January 2017 | Revised: 31 May 2017 | Accepted: 23 August 2017

DOI: 10.1002/rob.21757

REGULAR ARTICLE

WILEY

# An architecture for robust UAV navigation in GPS-denied areas

Francisco J. Perez-Grau[1] | Ricardo Ragel[2] | Fernando Caballero[2] | Antidio Viguria[1] | Anibal Ollero[2]

[1]CATEC, Center for Advanced Aerospace Technologies, Sevilla, Spain

[2]Department of System Engineering and Automation, University of Seville, Sevilla, Spain

**Correspondence**
Francisco J. Perez-Grau, Department of Avionics and Systems, CATEC, Center for Advanced Aerospace Technologies, Wilbur y Orville Wright, 19, 41309, La Rinconada, Sevilla, Spain.
Email: fjperez@catec.aero

## Abstract

This article presents a software architecture for safe and reliable autonomous navigation of aerial robots in GPS-denied areas. The techniques employed within key modules from this architecture are explained in detail, such as a six-dimensional localization approach based on visual odometry and Monte Carlo localization, or a variant of the Lazy Theta* algorithm for motion planning. The aerial robot used to demonstrate this approach has been extensively tested over the past 2 years for localization and state estimation without any external positioning systems, autonomous local obstacle avoidance, and local path planning among other tasks. This article describes the architecture and main algorithms used to achieve these goals to build a robust autonomous system.

## 1 | INTRODUCTION

Unmanned aerial vehicles (UAVs) have been the target for extensive research activities in the past several years. Their use in civil and commercial applications is continuously growing as they are becoming more popular, even essential in some cases such as emergency services. Outdoor obstacle-free operation based on GPS can be generally assumed for UAVs since different available commercial products already offer a mature performance, such as the mass-produced models from DJI.* However, some applications require the aerial vehicles to fly in confined environments where GPS might be unreliable or even unavailable. Relying exclusively on GPS for positioning might also pose safety issues in terms of signal jamming or platform hijacking, resulting in a potentially vulnerable system. The introduction of aerial robots collaborating with humans in different applications is also a strong motivation for procuring reliable autonomous systems that can operate safely in their functioning. Hence, there is still a need for reliability in several key enabling technologies to achieve a safe and robust autonomous system, such as localization, navigation, and planning.

To simplify the accomplishment of all the tasks related to the autonomous navigation of an aerial robot, a framework for combining and executing heterogeneous software modules has been developed. There are several open-source projects that target complete software architectures,[1] such as ArduPilot † or PX4‡. However these approaches usually lack flexibility to perform and support high-level functionalities, which are often demanded by users that wish to develop innovative approaches toward UAV autonomous navigation. In recent years, several initiatives have arisen from some research groups. The Autonomous Systems Lab from ETH Zurich developed *asctec_mav_framework*§; however, it is dependent on the UAV manufacturer such as shown in some of the lab members' work.[2,3] The Paparazzi project¶ from ENAC is a complete solution that also includes specific hardware, the Paparazzi autopilot. Another example is *hector_quadrotor*# from TU Darmstadt, which is heavily focused on simulation environments and has limited applicability in real systems performing experimental flights. Other works propose architectures for autonomous operations,[4–6] but there is no evidence of versatility regarding the application of the same approach in different platforms or using different sensors, since there are some architecture modules highly hardware dependent. Even though these approaches have produced major advances in this line of research, there are still important challenges related to the achieved level of autonomy and flexibility of the system to be able to adapt it for different aerial robots or applications.

Figure 1 shows our software architecture. The system is decomposed into several modules, which are in charge of different tasks. The core idea is the combination of simple concepts and components to build a reliable system. Each module runs as an independent process on-board the UAV. The different modules can implement a wide variety of algorithms and techniques for performing their task, according to the equipment of the aerial robot. It is important to point out that working with UAVs involves additional constraints in terms of computation and payload capacity which must be taken into account for the development of applicable systems. That said, the different algorithms

* http://www.dji.com/products/drones

† http://ardupilot.org/

‡ http://px4.io

§ http://wiki.ros.org/asctec_mav_framework

¶ http://wiki.paparazziuav.org/wiki/Main_Page

# http://wiki.ros.org/hector_quadrotor

 © 2017 Wiley Periodicals, Inc. 

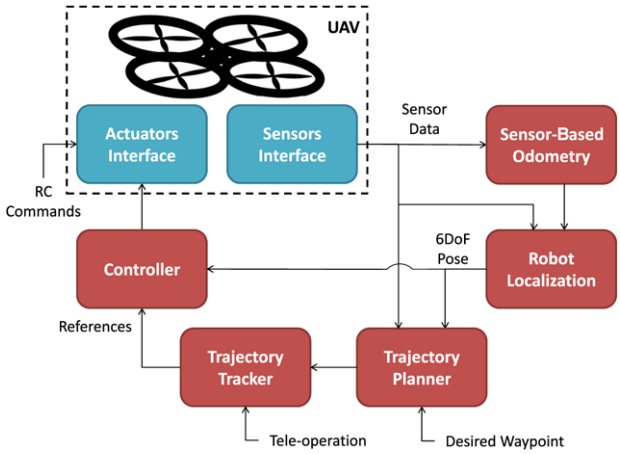



**FIGURE 1** Schematic overview of the proposed architecture

can be developed independently and switch or replace them within any of the modules to solve the task that is intended to perform. This does not affect the rest of the architecture provided that the new algorithm complies with the applicable input and output interfaces of the specific module.

Besides, the architecture is focused on the software components, remaining independent from the hardware used underneath. The core algorithms of the system do not depend on the specific manufacturer of the UAV platform or the specific sensors that are used, and hence it is possible to replace them with a different model or even with an upgraded version of the device, implying only minor changes to the system. Apart from that, it is possible to complement the system with additional sensors or enhanced processing capabilities without altering the overall scheme. Moreover, this architecture is able to accommodate applications involving a single or multiple UAVs, as well as highly autonomous operations or teleoperated missions.

One of the first things to consider for an autonomous robot is the estimation of the current state (position and orientation) of the platform in a specific coordinate system. For this, sensor data are first processed and used to obtain a localization estimation with respect to its environment. This is commonly carried out by an odometry calculation in *sensor-based odometry* and a localization estimation in *robot localization* to produce a reliable 6 degree-of-freedom pose (position and orientation). Sensor data are commonly used in both modules to reduce uncertainties and improve the robustness of the estimations, especially for long-term operation of the aerial robot.

It is very common to find approaches that fuse inertial sensors and altitude sensors (like barometers, ultrasonic sensors, or lasers) with monocular vision to estimate the robot localization based on visual simultaneous localization and mapping (SLAM).[7–10] These approaches work very well when we repeatedly visit the same area, but usually fail at high-speed UAV motions in unknown scenarios, although some recent works provide good performances under fast motions.[11] Optical flow approaches[12,13] make use of the same sensors but aim to estimate only linear velocities. These approaches have demonstrated to work very well for up and down facing cameras in UAVs when the viewed scene is approximately planar. This is a good solution when either the ground or the ceiling is planar, and it has enough texture, however, velocity integration for localization (odometry) quickly diverges, and these methods cannot be used for long-term localization.

Typically at the cost of higher computational requirements, stereovision systems provide direct depth measurements in a general three-dimensional (3D) environment for a given camera baseline. SLAM[14] and pure odometry approaches[15] demonstrate good results at short-term localization and even in long trajectories without loop closing.[2] Many of the algorithms developed for stereovision odometry and SLAM can be applied to 3D cameras based on pattern projection,[16,17] also using the recently available Google Tango.[18]

The approach followed by this research work splits the robot-positioning problem into odometry and localization. Some modules of the software architecture like the local planner or the controller will need a smooth and stable robot motion estimation; they do not care about the global positioning actually, instead they just need to know accurately how much the robot moved in the last seconds. Other modules like the global planner need accurate robot positioning into a map to plan the proper actions. We can find approaches in the state of the art that compute both estimations (odometry and localization) at the same time such as RGBD-SLAM,[16] ORB-SLAM,[19] CCNY RGB-D,[20] RTAB-Map,[21] or iSAM.[22] However, these approaches build their maps online and do not exploit the fact that the robot moves in a partially known environment; they are very useful for building such map off-line however.

Once we have a reliable robot localization, the next step will consider tackling more complex tasks, such as where to go or how to reach a specific location. The generation of safe trajectories is another essential capability for autonomous navigation, especially in cluttered areas where several obstacles or even other robots can be found. Other scenarios might involve health and safety regulations that must be followed to collaborate with human workers, such as in manufacturing plants. These environments exhibit fixed or mobile elements and simple or complex obstacles, all of them sharing the same area. Therefore, the aerial robot must continuously conduct a local recalculation of the trajectory in real time to avoid collisions with different types of elements as the on-board sensors gain information about its surrounding. Additionally, the UAV might also need to adopt different collision avoidance strategies depending on the nature of the obstacles; the robot should avoid persons differently than static objects for instance. Sensor data are also acquired by the *trajectory planner* for reactive collision avoidance, along with the localization output to compute safe trajectories in the aerial robot environment.

Planning UAV trajectories in a full 3D environments is a complex and time-consuming task. Many different approaches have been proposed in the state of the art for online and off-line UAV trajectory planning. However, if we consider the resource limitations (in terms of time and computation) of the aerial robot, two are the main methods used: probabilistic or sampling-based approaches and heuristic or graph-based methods.

Sampling-based algorithms, like Rapidly-Exploring Random Trees (RRT)[23,24] and Probabilistic Rad Maps (PRM),[25] are specifically designed to handle non-holonomic constraints (e.g., wheeled robots), high degrees of freedom, and large spaces that require to be rapidly

15564967, 2018, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/rob.21757 by Indian Institute Of Technology, Wiley Online Library on [28/09/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

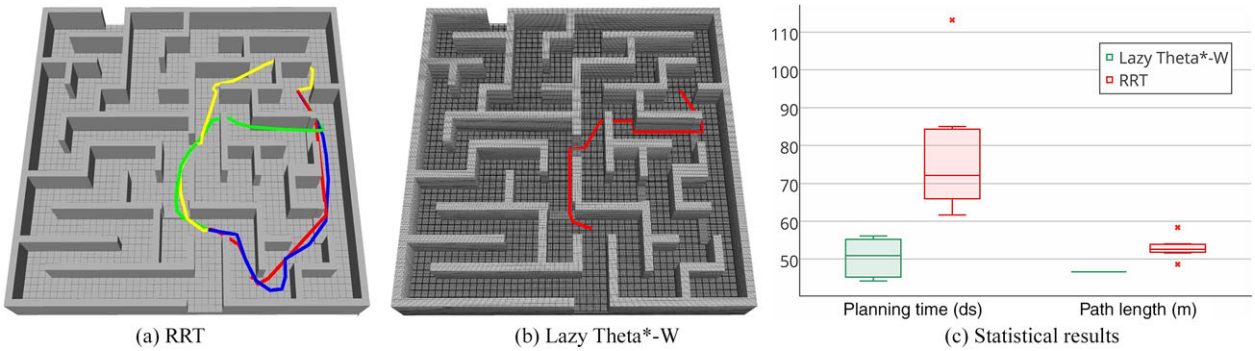



**FIGURE 2** Path planning results of RRT algorithm and our modified Lazy Theta* in a simulated 50 × 50 × 5 m maze, executed on a 2.0 GHz Intel i7 CPU. It can be seen how the trajectory computed by RRT greatly varies from one execution to another (represented through different colors), whereas the result of our modified Lazy Theta* is always the same

and uniformly explored, for example, a big manufacturing plant. Several variants of the RRT-based method have been proposed. RRT*[26] produces very optimal paths at the expense of non-real-time rates. On the other hand, RRT-Connect[27] resolves the timing problem, achieving faster solutions, but at the cost of obtaining longer paths. In Ref. 28. different probabilistic methods are used to solve the motion planning problem in UAVs. It proposes to use a continuous-time trajectory optimization algorithm to perform the local replanning. Their experimental results show that the replanning only took few milliseconds, meanwhile RRT* took some seconds. However, both algorithms present a significant inconvenience to be used as real-time algorithm in replanning tasks on UAVs: given an initial and target positions, the planned trajectory can vary significantly from one execution to another, so several consecutive replannings could result in unnecessary longer and inconsistent replanned trajectories. This can be easily seen in Figures 2(a) and 2(c), where the trajectories have been computed using the RRT implementation in *MoveIt!*.[29]

On the other hand, heuristic algorithms like A*,[30] Theta*,[31] and D*,[32] are specially designed to obtain the shortest path, exploring directly from the initial state to the target state, restricting the explored areas and getting a run time that is highly configurable and dependent on the number of variables and their resolution. The usual drawback imposed by discrete search techniques is that paths are formed by grid edges, so they are often not the shortest path in the continuous space. However, this issue was solved by the any-angle path planning Theta* and its Lazy Theta* variation,[33] which also optimizes the computational load of the algorithm. An additional advantage of graph-based algorithms is the possibility of reusing previous computation for replanning easier than sampling-based algorithms due to the computational advantages that graphs presents with respect to trees.[34] As shown in Figures 2(b) and 2(c), given an initial and target positions, the planning result of our variation of Lazy Theta* is close to the shortest path; it is always the same solution, and it gets always shorter paths and computation time than the RRT algorithm. This high repeatability of the results makes heuristic algorithms especially suitable for use as trajectory replanning methods, where several consecutive and connected paths are computed.

Thus, given the holonomic aerial robot motion characteristics, the limitations in terms of computation and the reduced planning space (just 3D positions), this paper opts for graph-based algorithms for planning trajectories, as it will be explained in the corresponding section.

The *trajectory tracker* ensures the UAV reaches the desired waypoints, calculating the reference positions, speeds, and times when they have to be commanded and thus sends the *controller* the references that it needs to calculate the necessary actions for the UAV. Special focus has been given to safety in the operation of the aerial robot; hence, the key components in our approach are the perception of the environment for both localization and motion planning. This will allow the UAV to select an appropriate behavior according to the awareness of the current situation. The state estimation (output of *robot localization*) and the commanded references (output of *trajectory tracker*) are sent to the *controller*. The robot state includes position, velocity, and attitude.

The context in which the authors have developed and tested the proposed architecture is related to the demonstration of high-level semiautonomous operation of a UAV for an inspection task. The motivation of this work is to enable unskilled workers to perform complex inspection missions with the aid of UAVs. This has never been possible before as the complexity of UAV requires expert piloting skills. In this context, we have developed a framework to perform localization and state estimation of the UAV without any external positioning systems (e.g., GPS or motion-capturing systems), perform autonomous local obstacle avoidance, local path planning, following of structures or homing of the aerial robot.

The paper is structured as follows: Section 2 describes the visual odometry system we have used in the experiments. Section 3 details the robot localization approach valid for GPS-denied areas. The different motion planning strategies we have integrated are described in Section 4. The field experiments are presented in Section 5, along with data from flight demonstration tests, followed by lessons learned and conclusions of this work.

## 2 | VISUAL ODOMETRY

This section focuses on a primary estimation of robot poses based on data obtained from an RGB-D camera, which provides images and 3D point clouds from its environment as shown in Figure 3. Our approach

15564967, 2018, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/rob.21757 by Indian Institute Of Technology, Wiley Online Library on [28/09/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

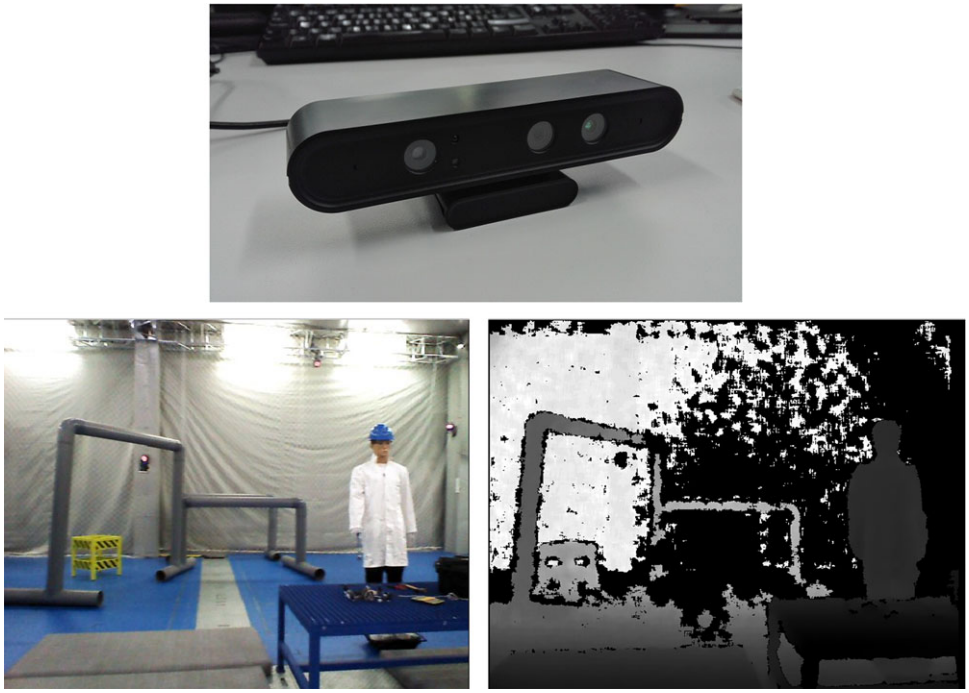


**FIGURE 3** The RGB-D sensor used in this work, an Orbbec Astra, along with sample RGB (left) and depth (right) images of the current scene in front of the UAV

follows the key ideas proposed in other approaches like rtSLAM,[35] LIBVISO,[36] or LIBFOVIS[37] but with special focus on computational resources and robustness. Our odometry system combines images, point clouds, pose estimation, and key framing in a loose-coupling filter to estimate a reliable and accurate localization at short term.

The approach in this work is based on a pipeline that takes advantage of all the information that the RGB-D sensor provides, both visual and depth data. The algorithm is similar to the previous author work detailed in Ref. 38, which is publicly available.* However, the previous work was based on a stereo camera, and hence the pipeline has been adapted to meet the particularities of RGB-D data. Besides, some improvements have been included, such as a fine tuning for the key-framing approach, the introduction of the inertial measurement unit (IMU) data as initialization for solving pose increments, and the computation of a sparse bundle adjustment for pose refining.

Our approach starts with a set of visual features extracted from the RGB images. At each frame, a set of interest points, or key points, is found from the RGB image. These key points are visual features extracted using the FAST algorithm (Features from Accelerated Segment Test),[39] which is a method for corner detection with a strong computational efficiency. The identified features contain local information that ideally make them repeatable across consecutive frames.

Depending on the environment, the scene in front of the UAV could be featureless, or there might be strong-detailed regions where all the key points are detected. Both cases pose problems in the subsequent stages due to poor transformation estimations. To overcome these issues, an image bucketing has been adopted. The image is divided into several regions (six buckets in our approach), and a minimum number of features is required to be extracted from each one, providing an even distribution of selected features.

The associated depth of each feature is obtained directly from the RGB-D sensor, thus enhancing the set of FAST features (two-dimensional [2D] key points) with depth information (3D key points). Next step involves finding correspondences between the sets of surviving key points from the current and the previous frames. To do that, the key points need to be described in such a way that some kind of comparison can be made among them, and for that the ORB descriptor (Oriented FAST and Rotated binary robust independent elementary features (BRIEF) )[40] is used. Our work in Ref. 38 used the BRIEF descriptor, but ORB exhibits two important benefits: it is rotation invariant and selects optimal sampling pairs of pixels around the key-point (BRIEF uses random sampling pairs).

To match the 3D key-point descriptors, a robust matcher has been developed. This matcher only considers correspondences that are symmetrical, that is are found from the current frame to the previous frame and vice versa. Perspective constraints based on fundamental matrix are also imposed to remove outliers. Once enough robust key points have been found, the set of matches is used to solve a perspective-*n*-point problem (P*n*P).[41] Given $N$ 3D points in coordinate frame $A$, and their 2D projections in a calibrated camera (with a known matrix of intrinsic parameters $K$), the P*n*P problem is to estimate the camera pose (rotation $R$ and translation $T$) relative to $A$. The 3D points correspond to the previous frame, and the 2D points are their projections in the current frame. Hence, the resulting pose constitutes the motion of the camera in the interval of time between both frames.

Odometry suffers from cumulative errors; therefore, a key-framing approach has been adopted to partially mitigate such an effect. New key frames are produced when the feature tracking flow exceeds a

* http://wiki.ros.org/viodom

15564967, 2018, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/rob.21757 by Indian Institute Of Technology, Wiley Online Library on [28/09/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

given threshold. The key frame stores information about its image descriptors, their 3D positions, and the aerial robot pose at that moment. Then, the frame matching occurs comparing the current frame against the last created key frame, instead of the immediately preceding frame, in such a way that cumulative errors are present exclusively within the production of new key frames.

The odometry system reads data from the on-board IMU, which is placed in the UAV autopilot, and integrates the roll and pitch angles with the purely visual estimation in a loose-coupling manner. Another feature that was developed to improve the robustness of the odometry estimation is a refinement of the poses using sparse bundle adjustment (SBA).[42] Considering the heavy computational weight of a full bundle adjustment problem, which implies the reconstruction of the 3D structure and motion, our approach only focuses on the camera poses (motion), leaving the structure as is by taking advantage of the quality of the 3D point clouds that the RGB-D sensor provides. We have implemented a lightweight approach that only considers the refinement of the six-dimensional (6D) poses (position and orientation) for the last $N$ key frames, speeding-up the algorithm and allowing online computation. The final pose estimation is transformed from the RGB-D frame to the aerial robot body frame and then is ready to be used by subsequent modules in our framework.

The aforementioned approach is the one we have used in the field experiments for demonstrating how our framework works. Nevertheless, any other odometry approach based on visual information or any other sensor data (e.g., LIDAR or radio-based ranging) can be used and adapted to meet the interfacing requirements of our architecture, which is based on standard robot operating system (ROS) data types.

## 3 | ROBOT LOCALIZATION

This section focuses on the robot localization in GPS-denied areas, relying only on information gathered by on-board sensors. As it was previously mentioned, visual odometry can deliver good pose estimations in the short term, but it will accumulate errors over time depending on the environment. Reliable long-term localization is a challenging problem that cannot be robustly solved using pure visual odometry approaches. Our architecture is mainly focused on the combination of visual odometry with additional sources of information to keep the localization errors bounded.

Our framework allows the integration of any custom solution for aerial robot localization, even if it simply relies on the pose estimation provided by the visual odometry module. The only requirement is to keep the data types for the interfaces with the rest of modules, especially the motion planning stack. The experiments described in this paper are based on the combination of visual odometry with an extension of a Monte Carlo localization (MCL)[43] for 6D localization using 3D data from the RGB-D sensor. This section details the development of such solution.

The use of an RGB-D sensor is a great advantage due to the fact that it is less reliant on the scene texture, unlike stereo cameras, and lighting conditions while operating indoors. These factors make RGB-D sensors more suitable for reliable localization in our framework. In our work, this sensor has been used to build a map of the environment where the UAV is going to operate. This is a common situation for many autonomous robot applications where the missions take place in a partially known environments; this map is required if the end-user wants to perform waypoint navigation. The use of a previously built map usually helps obtaining a reliable localization estimation thanks to the prior knowledge of the environment. This map can be precisely computed off-line using cameras or range sensors (e.g., LIDAR), using existing techniques.[16,44]

Our approach makes use of MCL, which is a popular technique for indoor robotics. It is based on a known map of the area and a particle filter to estimate the pose of the aerial robot within the map. We have implemented an enhanced version of the adaptive MCL algorithm that is available in the ROS wiki,* since this one is limited to ground robots moving in a 2D environment and equipped with a 2D laser scanner. An extension for 6D localization was presented in Ref. 45, which considers a 3D environment but it is still based on 2D laser scanner meant for 2D motion of humanoid robots; hence, it is not suitable for aerial robots. Our algorithm integrates 3D point clouds in a 3D map to solve the aerial robot localization. The short-term pose estimation is reliable thanks to the visual odometry input, and the localization errors over time are bounded thanks to the matching of point clouds against the map. The particle filter consists of $N$ particles $\mathbf{p}_i$, each one with the following state vector:

$$\mathbf{p}_i = [x, y, z, \psi]^T \tag{1}$$

where $\psi$ refers to the yaw angle of the aerial robot. Roll and pitch angles are not included into the state vector because they are delivered and observable from the UAV on-board IMU. This greatly reduces the computational complexity of the algorithm, allowing for real-time on-board computation.

The particles can be initialized manually or automatically by setting an initial position together with a covariance matrix to distribute the particles in the space, as Figure 4 depicts. To be able to perform autonomous navigation of the UAV from takeoff, the initial position and yaw angle are known. Otherwise, the filter would need to wait until the robot moves and the particles start moving to more likely pose hypotheses. Even though the convergence from automatic initialization does not take long, it is not adopted in our experiments for practical reasons.

The particles are propagated following a robot motion model based on a fast and reliable visual odometry computed from RGB-D data, as detailed in Section 2. Every time the algorithm receives sensor readings (i.e., RGB-D point clouds), each particle evaluates its accuracy by checking how likely it would receive such sensor reading at its current pose. Then the particles are redistributed (resampling) to new poses, which are more likely to be accurate. This iterative process involves moving, sensing, and resampling, whereas the set of particles should converge to a single cluster near the true pose of the robot. For that, weights are

* http://wiki.ros.org/amcl

15564967, 2018, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/rob.21757 by Indian Institute Of Technology, Wiley Online Library on [28/09/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

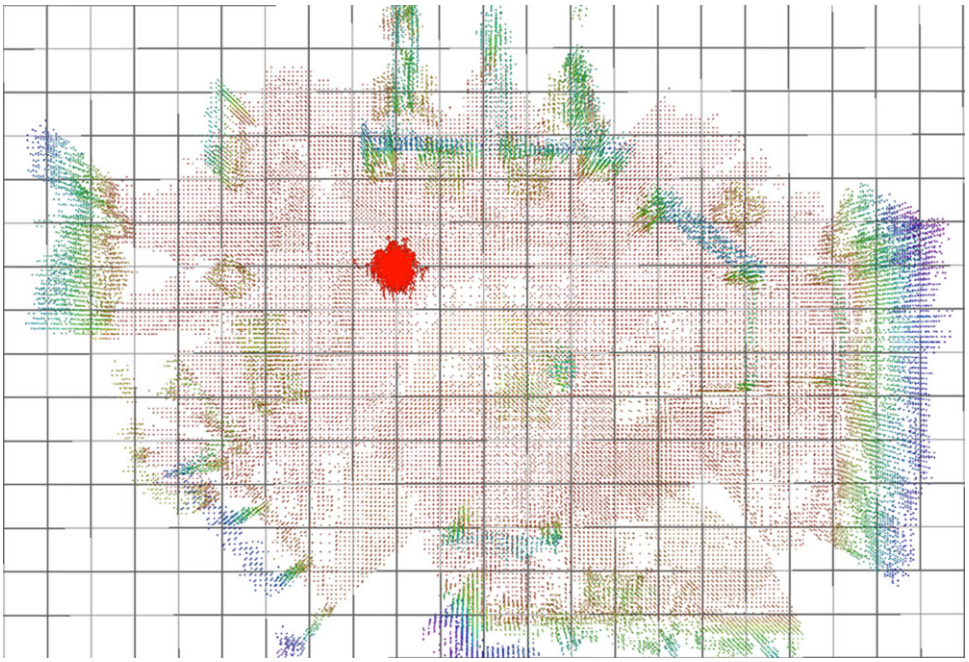


**FIGURE 4** Initialization of the particle cloud in a known position within the 3D map

computed for each particle according to its accuracy. Each particle has an associated weight $w_i$ such as:

$$\sum_{i=1}^{N} w_i = 1 \tag{2}$$

The filter prediction takes into account the increments provided by the odometry module for the robot state vector:

$$\Delta \mathbf{x} = [\Delta x, \Delta y, \Delta z, \Delta \psi]^T \tag{3}$$

Such increments are employed to move the poses of the particles according the following expressions:

$$x_i^{t+1} = x_i^t + \Delta x * \cos\left(\psi_i^t\right) - \Delta y * \sin\left(\psi_i^t\right) \tag{4}$$

$$y_i^{t+1} = y_i^t + \Delta x * \sin\left(\psi_i^t\right) + \Delta y * \cos\left(\psi_i^t\right) \tag{5}$$

$$z_i^{t+1} = z_i^t + \Delta z \tag{6}$$

$$\psi_i^{t+1} = \psi_i^t + \Delta \psi \tag{7}$$

The values of $\Delta x$, $\Delta y$, $\Delta z$, and $\Delta \psi$ are modified randomly following a normal distribution centered in their input values and with standard deviations which are proportional to each increment, for example, $\sigma_x = k_x * \Delta x$ with $k_x > 0$. The value of $k_x$ is a design parameter that depends on the accuracy of the visual odometry.

Furthermore, the filter update makes use of a map of the environment, which is provided as a 3D occupancy grid based on OctoMap,[46] as in Figure 5.

As stated before, subsequent received point clouds are transformed to the pose represented by each particle, and an alignment with the map is evaluated. Since the number of particles can be considerably high, and each RGB-D point cloud consists of around 300,000 points; the computational cost of performing this operation is significant. To reduce the computational load, we compute a 3D probability grid as in Refs. 45 and 47 in which each position stores a value of how likely it is that such position falls within an occupied point of the map, instead of storing binary information about occupancy as in the provided map. Each 3D position $p_i$ of the grid is then filled with probability values according to a specific Gaussian distribution centered in the closest occupied point in the map from $p_i$, $map_i$, and whose variance $\sigma^2$ depends on the sensor noise used in the approach.

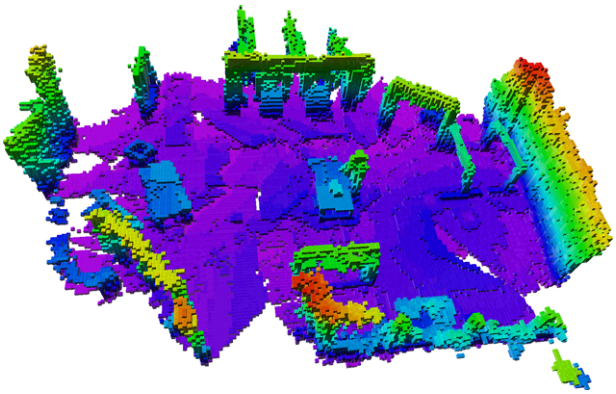


**FIGURE 5** The 3D map of the experiments scenario used as input for the MCL

$$\text{grid}(p_i) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-||p_i - \text{map}_i||^2/2\sigma^2} \tag{8}$$

Such a probability grid only needs to be computed once, is not required to be updated for a given environment, and relieves from performing numerous distance computations between each cloud point

15564967, 2018, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/rob.21757 by Indian Institute Of Technology, Wiley Online Library on [28/09/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

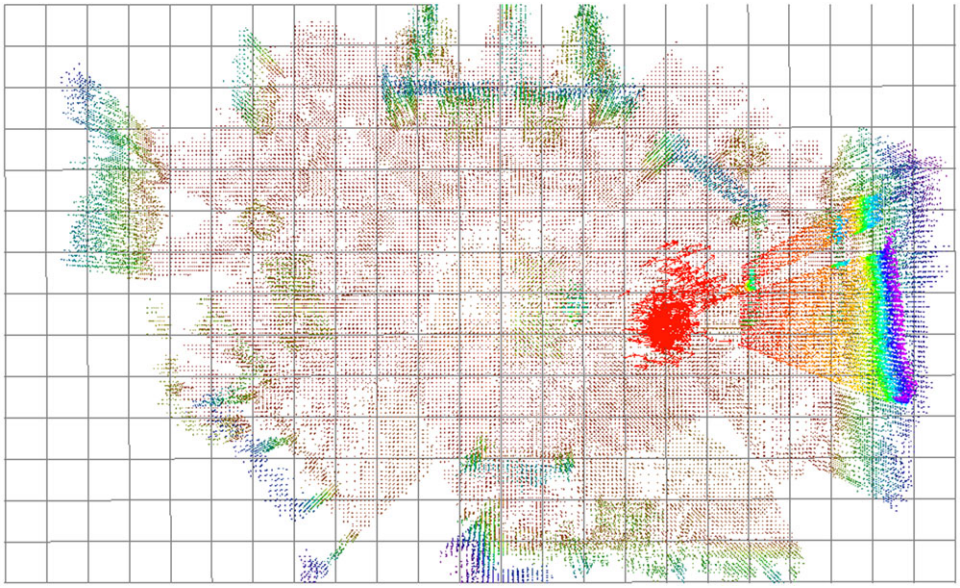


**FIGURE 6** Evolution of the particle cloud during the flight test and projection of the current 3D point cloud on the map from the estimated robot pose

for each particle and its closest occupied point in the map. Besides, each point cloud is first transformed according to the current roll and pitch provided by the on-board IMU. This transformation is done just once per update, reducing the computational requirements as well.

Then, for every point of the transformed cloud we access its corresponding value in the 3D probability grid. Such a value would be an indicator of how likely is that point to be part of the map. By doing this with every point of the cloud and adding all the probability values, we obtain a figure of how well that particle fits the true location of the aerial robot according to the map. Then, the weight $w_i$ of each particle $\mathbf{p}_i$ is computed. Assuming that the point cloud is composed by $M$ 3D points $\mathbf{c}_j$, the weight is computed by adding all the associated probability grid values as follows:

$$w_i^{\text{map}} = \frac{1}{M}\sum_{j=1}^{M} \text{grid}(\mathbf{p}_i(\mathbf{c}_j)) \tag{9}$$

where $\mathbf{p}_i(\mathbf{c}_j)$ stands for the transformation of the point to the particle's state, and $\text{grid}(\mathbf{p}_i(\mathbf{c}_j))$ is the evaluation of the probability grid in such transformed position.

Finally, the resampling stage spreads the particles to the maximum likelihood areas following the low variance sampler algorithm described in Ref. 48. The updated state vector for the aerial robot is then calculated as the weighted sum of all the particles. Figure 6 shows one of the final moments of the experiment, where we can observe the particle cloud and the associated point cloud projected from the calculated pose corresponding to the weighted sum of the particles.

Nevertheless, the described technique is not the only option to perform robot localization in our framework. Any user should be able to provide pose estimations to the subsequent modules of our architecture given they comply with the interfaces and data types. For example, another suitable approach can be based on the combination of the available visual odometry with radio-based ranging, as the authors already demonstrated in Ref. 38. These sensors provide distance measurements between them, which can be employed to bound localization errors. Moreover, such distance measurements could be perfectly integrated into the MCL filter update stage to combine the particle weights by using both the 3D point clouds and the distance measurements.

## 4 | MOTION PLANNING

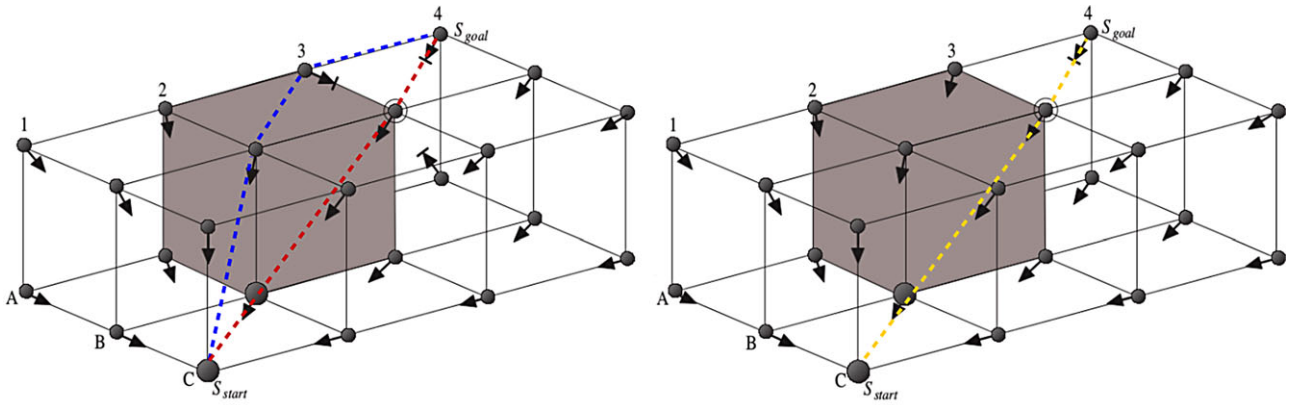
Once we have the UAV up and flying based on local perception, the next objective is planning and executing trajectories in partially known environments, so that we can perform waypoint navigation and obstacle avoidance efficiently.

As stated in the Introduction, the proposed global and local planners will be graph based and built upon a version of Lazy Theta*, which we call Weighted Lazy Theta*, or Lazy-Theta*-W. It is able to reduce the planning time, maximize as much as possible the probability of success, and always obtain the same replanned trajectory from an initial to a target position for each map. This paper extends the original weighted heuristic distance proposed by Daniel et al.[49] at two levels. First, this paper proposes weighting the heuristic distance in the opposite way to,[49] that is reducing the expanded vertexes to decrease the replanning time. Second, we also add weighting to the symmetric 3D search, focusing the exploration in a desired asymmetric volume and, therefore, further reducing the planning time.

### 4.1 | Weighted Lazy Theta*

The main disadvantage of A*-like algorithms is that paths are formed by grid edges, so they are often not the shortest path in the continuous space, so they need a postsmoothing to become realistic trajectories.[31] Theta* is built upon A*, and it does not constrain paths by graph edges; each expanded vertex performs a line-of-sight check[50] for each unexpanded visible neighbor that allows the parent of this vertex to be any

15564967, 2018, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/rob.21757 by Indian Institute Of Technology, Wiley Online Library on [28/09/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License




(a) Generated paths by A* (blue) and Theta* (red), and vertex parents from Theta* (arrows)

(b) Generated path by Lazy Theta* (yellow), and its vertex parents (arrows)

**FIGURE 7** Lazy Theta* evolution from A*

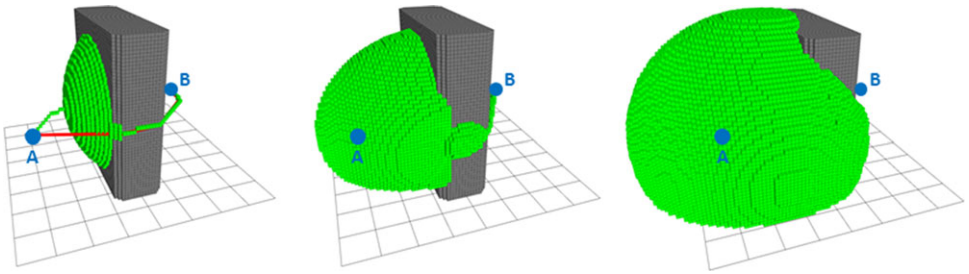



**FIGURE 8** Expanded Vertex (green) and Result Path (red) calculated by Weighted Lazy Theta* to go from point A to point B avoiding a wall. Goal Weight $w$ is modified from $w = 10$ (left), to $w = 1$ (center) and $w = 0.5$ (right)

other graph vertex, and thus it finds short and realistic looking paths. Unfortunately, the number of line-of-sight checks that Theta* performs in 3D space is very high, increasing the computational requirements. Lazy Theta* is introduced as a simple improvement in the Theta* algorithm that reduces the number of line-of-sight checks, finding paths faster than the original algorithm (thus reducing the computation load) without an increase in path length. Examples of paths found by A*, Theta*, and Lazy Theta* are shown in Figure 7, where vertex parents for Theta* and Lazy Theta* can also be seen.

The main properties of our proposed Lazy Theta*-W are the following:

- It reduces the planning time even further. Even though the *lazy* variant greatly decreases the algorithm run time, the planning time could be excessive for replanning if, for example, large obstacles are present. This is caused by the symmetric 3D expansion of vertexes along the obstacle while searching for the solution path, as it can be seen at the center of Figure 8.
- It deals with restrictions imposed by the physical location of the optical sensor. In our platform, there is only one optical sensor placed along the *X*-axis and pointing forwards with a small tilt angle, so it does not provide information from neither behind nor above the UAV.

Thus, the Lazy Theta*-W algorithm variation is based on weighting the heuristic distance as follows:

$$h(\mathbf{s}) = w \cdot c(\mathbf{s}, \mathbf{s}_{goal}), \text{ with } w \in \mathbb{R}^+ \quad (10)$$

where this $h$-value is the straight line distance from a vertex $\mathbf{s}$ to the goal vertex $\mathbf{s}_{\text{goal}}$, which could be weighted with positive weights $w$. As it can be seen in Figure 8, where the start position is in front of a wall and the goal is behind the wall, if $w$ is less than one, the search expands more vertexes, so the algorithm would be able to get shorter paths at the expense of increasing the planning time. However, the possible path length decrease is not guaranteed and it would be only a slight reduction in comparison with the high run time increment, as it is shown in the experiments of Figure 10(a). Rather, if $w$ is greater than one, the expanded number of vertexes is reduced, greatly decreasing the run time and without implying a significant increase in the length of the resulting paths, as it can also be seen in those figures. These experiments show that a higher value, up to $w = 10$, is a good weight choice, obtaining a path planning time of 0.24 s (an 82.5% improvement from the original Lazy Theta*) for a path length increase of only 0.18 meters (just 2.3% longer than the original Lazy Theta* path).

The second improvement is focused on dealing with the sensor field-of-view restriction and a further reduction of the planning time. Unlike ground robots, aerial robots usually have restrictions in the number and type of on-board sensors to acquire environment information. Particularly, it is unusual to have 360° sensing coverage in UAVs. On-board sensors are commonly arranged so that the perception in the direction of flight is maximized. While such sensor distribution is a good trade-off between weight and perception, it might be harmful to perform local replanning without any restrictions, because the awareness in the vertical axis is usually reduced. Therefore, it is necessary to be aware of this limitation and take it into account in

15564967, 2018, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/rob.21757 by Indian Institute Of Technology, Wiley Online Library on [28/09/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

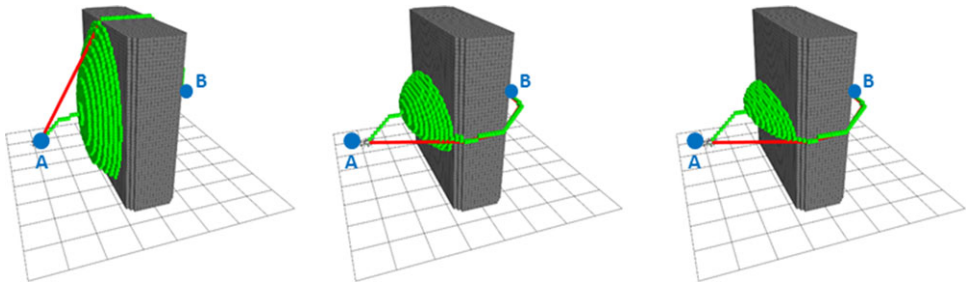
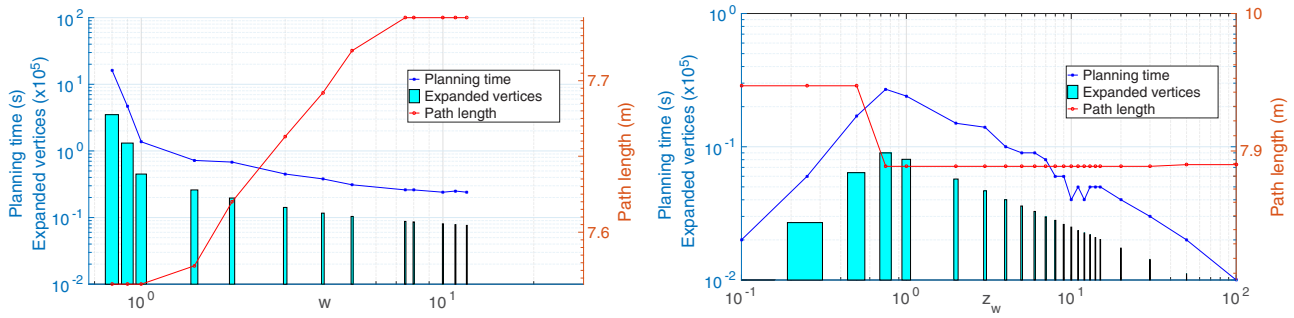



**FIGURE 9** Expanded Vertex (green) and Result Path (red) calculated by Weighted Lazy Theta* to go behind a wall. Height Weight $z_w$ is modified from $z_w = 0.5$ (left), to $z_w = 5$ (center) and $z_w = 20$ (right)




(a) Increasing the Goal Weight $w$ (b) Increasing the Height Weight $z_w$

**FIGURE 10** Path planning results for different values of parameters $z_w$ and $w$ in consecutive simulations on a 2.0 GHz Intel i7 CPU. The experiments consist of planning from current UAV position to behind a wall, with an algorithm resolution of 0.1 m

the path replanning, not entirely disabling height changes, but preferably choosing paths with small height variations. The proposed local replanner will prioritize the horizontal planning over the vertical one to address this issue.

The implementation is based on the first algorithm variation, but weighting the height axis with the straight line distance cost function $c(s, s_{goal})$ by the $z_w$ parameter as described in Eq. (11), where $\mathbf{s}_{goal} = [s_x, s_y, s_z]$ is the goal position. As shown in Figure 9, by increasing the height weight $z_w$, the *Z*-axis exploration is reduced, prioritizing the horizontal planning and, therefore, decreasing the algorithm run time without an appreciable path length increment, while enabling a possible vertical solution if a horizontal path is not found or too long. As it can be seen in Figure 10(b), for a height weight value $z_w = 5$, to which the *z*-axis is still enabled, the planning time obtained in the experiment is 0.09 s, a 62.5% improvement.

$$c(\mathbf{s}, \mathbf{s}_{goal}) = \sqrt{(s_{goal,x} - s_x)^2 + (s_{goal,y} - s_y)^2 + z_w \cdot (s_{goal,z} - s_z)^2}, \text{ with } z_w \in \mathbb{R}^+ \quad (11)$$

## 4.2 | Occupancy maps

The complete motion planning system handles two main different occupancy maps, the *global occupancy map* (GOM) for the global planner, and the *local occupancy map* (LOM) for the local planner. The GOM is built from an Octree[46] with known obstacles, whereas the LOM is updated in real time from the point cloud generated by the 3D camera. This point cloud is also preprocessed to remove the outliers and reduce the number of points to a specific algorithm resolution in order not to overload the local planner.

Both maps are voxel-based matrices used by the Theta* algorithms to obtain the paths. They are mainly defined by

- Voxels *resolution*. Voxels are the occupancy matrix elements, and their size determines the minimum distance between the possible states of the UAV. Too high values could make impossible intermediate needed positions, but if it is set too low the computation load would increase the planning time too much.
- Occupied points *inflation*. Input occupied points from the Octree (see Fig. 11 at the top) or from the point cloud (see Fig. 11 at the bottom) are inflated horizontally and vertically by half the UAV size, plus a safety factor. In this way, the algorithms are able to plan for the geometric center of the robot avoiding real obstacles and keeping a specific safety distance.
- *Workspace*. It is the matrix size and also represents the planner algorithm axis limits. Its limits are inflated by default to disable planning near the area boundaries, including the floor and the ceiling.

Unlike the GOM, which is considered permanent in the *world* frame, the LOM is dynamically built at the *local stabilized* frame (which is the same as the UAV frame but with zero roll and pitch) in each algorithm iteration from an *auxiliary occupancy map* (AOM). This AOM is at the *world* frame and has the same size as the GOM, but takes as input

15564967, 2018, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/rob.21757 by Indian Institute Of Technology, Wiley Online Library on [28/09/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

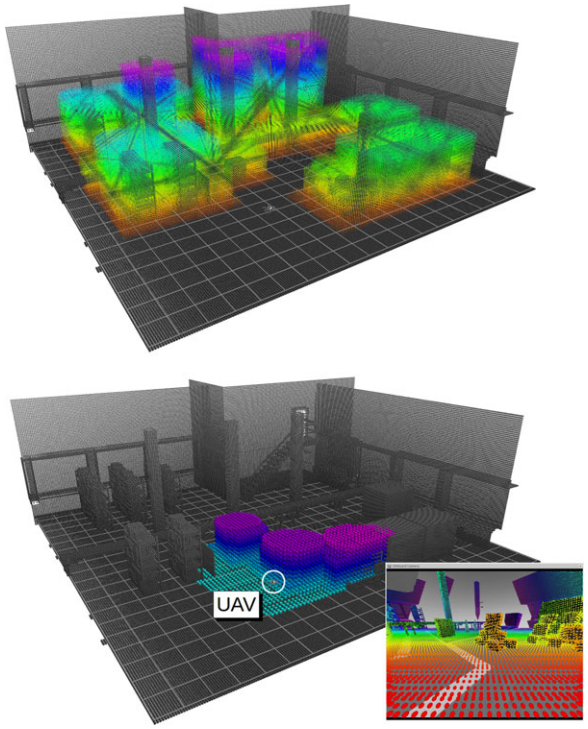



**FIGURE 11** Top figure shows the GOM (colored cubes) computed from an input Octree (gray cubes). Bottom figure shows the LOM (colored cubes) that is being built from the depth camera point cloud (colored spheres over the camera image at the bottom right) and the AOM, as detailed in Figure 12

the points from the preprocessed point cloud, which are buffered and removed according to the following two principles:

- The input point cloud will always be subject to outliers. Unlike LIDAR sensors, the quality of the depth information in RGB-D and stereo cameras depends on the imaged scene. Factors like texture, sunlight, or brightness might produce wrong depths that could be transferred into fake obstacles in the occupancy map.
- Unknown obstacles must be considered dynamic objects in general, so an occupied point cannot be permanent in the occupancy map.

For these reasons, the AOM elements are not just free or occupied voxels, but also include two counts: a *count to be accepted*, that is a point in the space needs to be perceived a minimum number of times before setting the corresponding voxel as occupied; and a *count to be removed*, that is an occupied voxel is kept a maximum number of times from the last time a corresponding point was seen before being removed. This dynamic behavior is shown in Figure 12, presenting a simplified flow diagram of the point cloud processing from the raw sensor measurements to the Lazy Theta*-W occupancy matrices. The *clear* box represents the release of occupied voxels around the UAV to decrease the possibility that the current robot state is within an occupied voxel. This is likely to occur when the trajectories are very close to the inflation of obstacles, such as in our proposed algorithm, and the transformation of the point cloud may not be consistent for different robot locations.

## 4.3 | Global planner

The global planner (*GP*) consists in a Lazy Theta*-W algorithm that computes the path from the current UAV position (or the nearest valid position) to a goal position (or the nearest valid position) in the GOM. This algorithm could be configured to replan optimally and with height changes ($w \approx 1$ and $z_w \approx 1$), because the global plan is only computed once per goal and, in principle, this planning time is not critical. Also, this path is postprocessed to get a global trajectory from the current UAV location to the final goal location.

The global trajectory is computed from the obtained path based on chosen linear and angular (*yaw*) mean velocities, a specific maximum distance between waypoints (*wp*), and a desired attitude reference evolution that allows the depth camera to track all possible obstacles along the path. The latter is maybe the most important factor because the robot only has one front sensor, so it needs to look ahead in the direction of movement as long as possible to ensure a safe global trajectory.

As shown in Figure 13, two yaw tracking methods have been implemented. The first one, called *yaw ahead at time*, is the simplest option: command to turn around the yaw axis at the waypoint $wp_i$ in the direction to the next one $wp_{i+1}$, setting yaw ahead from $wp_i$ to $wp_{i+1}$ during the segment $\mathbf{s}_{i\to i+1}$. Given that the successful performance of the sensor requires slow yaw rotation rates, and that the camera is pointing at the flight direction as long as possible, this method is not the safest solution because it may generate blind movement segments. The second implementation, called *yaw in advance*, tries to preempt this last possibility of collision. It computes the yaw reference of the waypoint $wp_{i-1}$ as the direction from $wp_i$ to $wp_{i+1}$, setting the yaw ahead from $wp_i$ to $wp_{i+1}$ during the segment $\mathbf{s}_{i-1\to i}$. This may give the idea that it could also have inconveniences because the robot would try to focus a waypoint before it has achieved the previous setpoint. However, during the previous segment, the sensor has already seen this last setpoint so that problem does not really exist.

The path segmentation has two objectives. The first one is to guarantee that the trajectory waypoints are close enough to each other so that time references, computed from mean velocities, and attitude references are realistic. In this way, it is possible to compute yaw

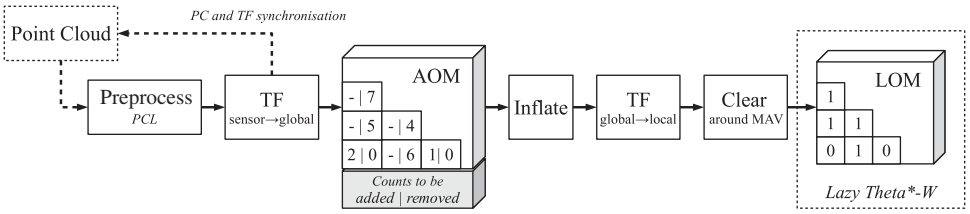



**FIGURE 12** Occupancy maps flow from the raw point cloud to the Weighted Lazy Theta* occupancy matrix

15564967, 2018, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/rob.21757 by Indian Institute Of Technology, Wiley Online Library on [28/09/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

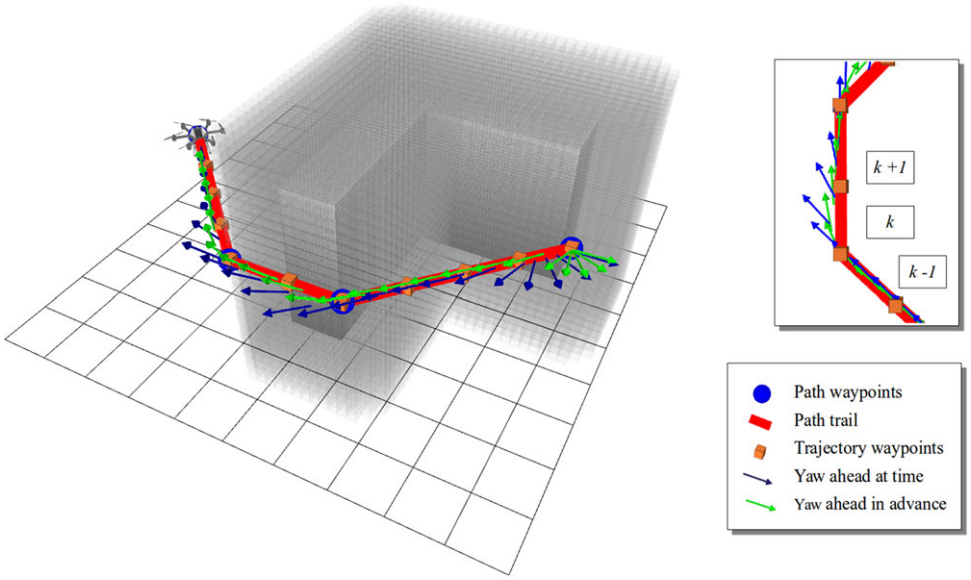



**FIGURE 13** Global planning and result trajectory using the two implemented yaw tracking methods. The resulted attitudes are shown as arrows

references at close positions that enable safe tracking of possible obstacles along the path. The second objective is to have the origin and the goal of a potential re-planning as close as possible to each other. As a result, the original global trajectory is less modified after several replanning operations, and the local replanning time is shorter.

As noted above, if the initial or the goal positions are occupied voxels, the planner will not be able to compute a path, so it is necessary to find the nearest neighbor of the occupied position. Such search is carried out from the nearest to the furthest voxel layer on the discrete square pyramid (Johnson solid $J_1$) with base centered at the occupied voxel, as shown in Figure 14.

Finally, the global planner implements a state machine over the local planner and the trajectory tracker to ensure that the goal is reached or to stop the trajectory execution if the replanning fails a given number of times.

## 4.4 | Local planner

The local planner (*LP*) is a two-step procedure that handles the global trajectory according to the LOM information, as illustrated in Algorithm 1. First, it checks if replanning is really necessary (*checkReplan*() function) due to the presence of an obstacle, provided by the point cloud, intersecting the global trajectory. Otherwise, the trajectory is free of collisions and its execution can continue. Second, if replanning is necessary, the *replan*() function is executed to try to avoid the newly detected obstacle by getting a new and safe global trajectory. Figure 15 presents two replanning cases during the same global trajectory execution. In the first case (at the left of the figure), the replanning is caused by an object which was known but wrongly positioned in the GOM, and hence was incorrectly avoided by the global planner. Nevertheless, the sensor point cloud is used to detect and correctly relocate the obstacle in the LOM, and the local planner can safely avoid it. It must also be noted that the local planner tries to replan following the original global trajectory, by only changing the trajectory segment affected by the new obstacle. In this case, the planner uses local information because the GOM might have been modified or wrongly positioned due to localization errors. In the second case (at the right of the figure), there is another object along the way, which is completely unknown to the global planner, but the local planner avoids it smoothly.

### Algorithm 1: Local planner main loop

```
Data: PointCloud, RobotState and GlobalTrajectory
Loop
    if New GlobalTrajectory then
        LP ⇐ GlobalTrajectory
        send(GlobalTrajectory)
    if GlobalTrajectory → Waypoints > 0 then
        LP → updateAOM(PointCloud, RobotState)
        if LP → checkReplan() then
            if LP → replan() then
                GlobalTrajectory ⇐ LP
                send(GlobalTrajectory)
            else
                status(replanfailure)
    else
        LP → cleanAOM()
at replan rate
```

The *checkReplan*() and *replan*() functions are described in Algorithm 2. The first one consecutively checks if all the waypoints from the current robot position to the last waypoint inside the local workspace have line-of-sight* (*LoS*) with its predecessor. If at least one

* Actually, the local planner implements a *soft* line-of-sight check that also considers as valid the visibility from near voxels to the origin waypoint and near voxels to the target waypoint. In this way, it avoids several replanning operations due to possible point cloud noise.

15564967, 2018, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/rob.21757 by Indian Institute Of Technology, Wiley Online Library on [28/09/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

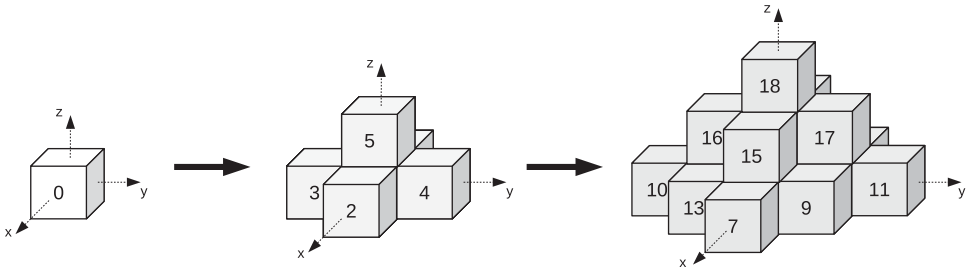



**FIGURE 14** Search progression from the occupied voxel to further voxel layers on a regular square pyramid

of these pairs of consecutive waypoints does not have *LoS*, replanning will be necessary. Then, the origin of the replanning will be the current robot position, and the target waypoint of the replanning will be the first one that has *LoS* with its predecessor after the interrupted segments, which is the first trajectory segment without obstacle (see again Fig. 15 and check the green lines).

The *replan*() function makes use of a fast Lazy Theta*-W to get a collision-free subtrajectory that replaces its corresponding trajectory segment, getting a new and safe global trajectory. Previous to the replan, this function checks whether the origin and the target of replanning are valid positions (nonoccupied voxels inside the local workspace). The origin is the current robot position, which could be

**Algorithm 2: Local planner replanning functions**

```
Data: RobotState, GlobalTrajectory and LocalWorkspace
checkReplan()
    LocalSubTrajectory ⇐ GlobalTrajectory
    wp_middle := wp_origin := RobotState
    wp_target := LocalSubTrajectory → First waypoint
    ExistReplanning := false
    while wp_target ∈ LocalWorkspace do
        if ∃ lineOfSight(wp_middle,wp_target) then
            if ExistReplanning then
                return true
        else
            ExistReplanning := true
        wp_middle := wp_target
        wp_target := LocalSubTrajectory → Next waypoint
    if ExistReplanning then
        wp_target := LocalSubTrajectory → Last waypoint ∈ LocalWorkspace
        return true
    else
        return false
end
replan()
    if wp_origin is occupied then
        searchNearValidPosition(wp_origin)
    if wp_target is occupied then
        searchNearValidPosition(wp_target)
    patch ⇐ ThetaStar(wp_origin, wp_target)
    if patch := NULL then
        return false
    else
        GlobalTrajectory ⇐ LocalSubTrajectory ∪ patch
        return true
end
```

15564967, 2018, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/rob.21757 by Indian Institute Of Technology, Wiley Online Library on [28/09/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

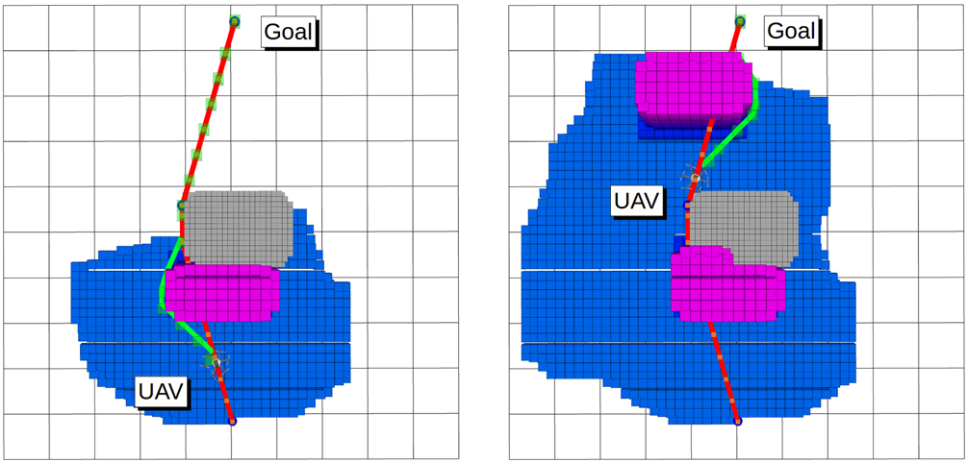



**FIGURE 15** Consecutive replanning subtrajectories (green line and green cubes) of the local planner along a global trajectory (red line and orange cubes) given two waypoints, one around the middle of the grid and another one at the top (marked as goal). GOM and LOM are shown in gray and blue/purple, respectively. Left image shows the UAV avoiding a known wall that is placed incorrectly in the GOM, whereas the right image shows the UAV avoiding a fully unknown wall

occupied, and the target could be, for example, the last waypoint inside the local workspace, which may be occupied too. Therefore, if one of them are not valid, it will search the nearest appropriated positions, using the same procedure as for the global planner shown in Figure 14. Finally, if *replan()* was executed, the new global trajectory is postprocessed to get the correct *yaw in advance* and time references for all its waypoints.

To achieve on-line replanning, the Lazy Theta*-W configuration must take into account the limited amount of time between perception and reaction to an a priori unknown obstacle and also the visual sensor field-of-view restriction. For these reasons, both the goal weight and the height weight parameters must be greater than one (e.g., $w = 10$ and $z_w = 4$) to achieve fast replanning times and limited height variation. Besides, the resolution of the LOM could be slightly increased, as it can be observed in Figure 15, to decrease replanning times even more.

As noted in sub-section 4.2 regarding occupancy maps, the points corresponding to detected objects are inflated to keep a safe distance between the UAV and the obstacles. However, such distance could be equal to the robot size plus just a few centimeters, which is not enough if the detected object is a human or another UAV. In the case of human detection, the local planner includes an infinite cylinder (along all workspace's *Z* axis) with radius of at least 1.2 meters from the detected human position in the *LOM*. The infinite *Z*-axis inflation makes impossible to replan above the person, while the safe distance radius ensures that the robot does not violate the human's personal space.[51] If another UAV is detected, which is usually in movement, the best way to avoid collisions is to go around its trajectory. In this case, the local planner also includes an infinite cylinder but in the estimated flight direction of the detected UAV, for example replanning over it if its trajectory is approximately horizontal.

## 4.5 | Trajectory tracking

Path tracking is usually referred to as the task to complete a path as fast as possible under the vehicle physical constraints.[52] In contrast, trajectory tracking classically requires that the vehicle is at a certain time at a certain position. The target of the trajectory tracking module is the latter, accomplishing that all trajectory waypoints $wp_i = [x_i, y_i, z_i, \psi_i, t_i]$ are safely reached, where $\psi$ refers to the yaw angle of the aerial robot and $t$ refers to the time from start in which the waypoint has to be reached (see Figs. 13 and 15). It guarantees that the input waypoints are reached within the specified times, as long as these positions and times meet the physical limitations of the robotic platform. If the physical limitations are not met, then the waypoints' positions will be reached but not at the specified times. There are some additional requirements that imply the following:

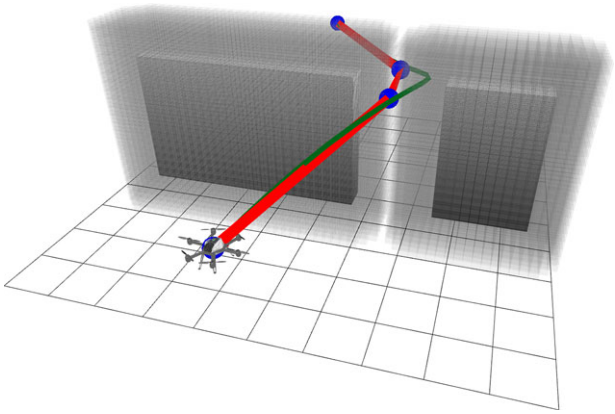


**FIGURE 16** Possible trajectories from the raw path (blue spheres) using splines interpolation (green line) and straight lines (red line)

- Time references must be the highest value among the minimum necessary time to achieve the required linear and angular displacement in each axis (*X*, *Y*, *Z*, and yaw). Hence, it ensures that we have enough time to reach the waypoint in all the axes. This is critical to make sure that the camera is pointing at the direction of flight as long as possible.
- The transitions between waypoints must be smooth and without overhead. As shown in Figure 16, the continuous trajectory can be

15564967, 2018, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/rob.21757 by Indian Institute Of Technology, Wiley Online Library on [28/09/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

generated using spline interpolation, but this does not guarantee to comply with safety distances with respect to obstacles. Following the straight line between waypoints is the only way to ensure that the UAV stays away from known or detected obstacles along the complete trajectory, because the solution of Theta* algorithm only guarantees the *LoS* between them.

For that purpose, we need a continuous trajectory generator from the discrete trajectory waypoints, and a trajectory executor to the robot controller. The trajectory generator computes velocity commands for each robot axis along each trajectory segment $\mathbf{s}_{i\to i+1}$ (from $wp_i$ to $wp_{i+1}$), according to the velocity at the beginning of the line segment $v_i$ (at $wp_i$) and based in a piecewise linear model of the UAV linear velocities:

$$v_i(t) = \begin{cases} v_i & \text{if} \quad t < t_i^{\text{ref}} + \tau \\ m(t - (t_i^{\text{ref}} - \tau)) & \text{if} \quad t_i^{\text{ref}} + \tau \le t \le t_i^{p} \\ v_i^{\text{ref}} & \text{if} \quad t > t_i^{p} \end{cases} \tag{12}$$

where $v_i^{\text{ref}}$ is the target value of the velocity reference along $\mathbf{s}_{i\to i+1}$, $t_i^{\text{ref}}$ is the instant when the change of velocity occurs, $t_i^p$ (*peak time*) is the instant at which $v_i^{\text{ref}}$ is achieved, $m$ is the estimated permanent velocity rate (the maximum acceleration), and $\tau$ is the velocity delay.

This model is integrated using Eq. (13) along all trajectory segments. owing to the fact that the change of velocity references is anticipated to each previous trajectory segment, this system does not depend directly on $t_i^{\text{ref}}$, but rather on $t_{i+1}^{\text{ref}}$. For the same reason, the velocity rates $m_i$ and $m_{i+1}$ (at $wp_i$ and $wp_{i+1}$, respectively) are computed as the slopes from $v_i^{\text{ref}}$ to the velocity at the start and at the end segment waypoints, respectively.

$$\begin{cases} x_{i+1} - x_i = \int_{t_i}^{t_{i+1}} v_i(t)dt & \text{with} \quad t_i < t_{i+1} \\ m_i = \dfrac{v_i^{\text{ref}} - v_i}{t_i^p - t_i} & \text{with} \quad t_i < t_i^p < t_{i+1}^{ref} \quad \text{and} \quad |m_i| = m \\ m_{i+1} = \dfrac{v_{i+1} - v_i^{\text{ref}}}{t_{i+1} - t_{i+1}^{ref} - \tau} & \text{with} \quad t_{i+1}^{ref} < t_{i+1} \quad \text{and} \quad |m_{i+1}| = m \\ v_i \approx \dfrac{v_{i-1}^{\text{ref}} + v_i^{\text{ref}}}{2} & \text{if} \quad \text{sign}(v_{i-1}^{\text{ref}}) = \text{sign}(v_i^{\text{ref}}), \ \text{else} \ v_i = 0 \\ v_{i+1} \approx \dfrac{v_i^{\text{ref}} + v_{i+1}^{\text{ref}}}{2} & \text{if} \quad \text{sign}(v_i^{\text{ref}}) = \text{sign}(v_{i+1}^{\text{ref}}), \ \text{else} \ v_{i+1} = 0 \end{cases} \tag{13}$$

The previous system presents five equations and five variables: $v_{i-1}^{\text{ref}}$, $v_i^{\text{ref}}$, $v_{i+1}^{\text{ref}}$, $t_{i+1}^{\text{ref}}$, and $t_i^p$, assuming that the velocity at the waypoints, $v_i$ and $v_{i+1}$, is known. Actually, these depend on velocity references except in the changes of direction (yaw angle), where they are set to zero for safety reasons. Because of this, we use an iterative method to solve the problem, initializing all the variables to an average value of the segments mean velocities ($\frac{\Delta x}{\Delta t}$) and updating them according to velocity references from the solution of the previous iteration (last two equations).

The final generated trajectory reference is composed by the $v^{\text{ref}}$ and $t^{\text{ref}}$ vectors. The execution is carried out by sending position and velocity commands ($p^{\text{cmd}}$, $v^{\text{cmd}}$) to the controller, which are computed linearly from $v^{\text{ref}}$ and the robot odometry in each trajectory segment and in each instant of time. In addition, rate limiters are applied to $p^{\text{cmd}}$ and $v^{\text{cmd}}$ to take into account the physical limitations of the robot and the desired smoothness in the movements.



**FIGURE 17** The AscTec Neo UAV used for the experiments

## 5 | FIELD EXPERIENCE

This section provides information about the experimental experience gathered by the authors over 2 years of testing. Prior to the discussion of the experimental results, the setup describing all the elements involved is presented in the next subsections.

### 5.1 | Architecture framework

Our approach uses as middleware the ROS,[53] which is widely used in research activities and is becoming a de facto standard for robotic applications. The interfaces between modules are based on standard ROS data types for transformations, pose estimations, or trajectory commands. This greatly improves the flexibility of the architecture to allow testing custom high-level algorithms for autonomous operation, while at the same time enables the use of UAVs from different vendors and a wide variety of sensors for which supported drivers are already available. We have made extensive use of the tools available in ROS to help debugging and detecting issues in both hardware and software. This includes simulations run in Gazebo, logging, and playing back data from different experiments or visualization and inspection tools. This is especially useful when an innovative approach or algorithm is under development, and there is a potential risk of crashing the platform. Thanks to the use of these tools, it is possible to test the algorithms and modules off-line and debug internal states of the UAV or present relevant data in a graphical interface.

### 5.2 | UAV platform

The UAV used to demonstrate our architecture is a research prototype from Ascending Technologies called AscTec Neo showed in Figure 17.

15564967, 2018, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/rob.21757 by Indian Institute Of Technology, Wiley Online Library on [28/09/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

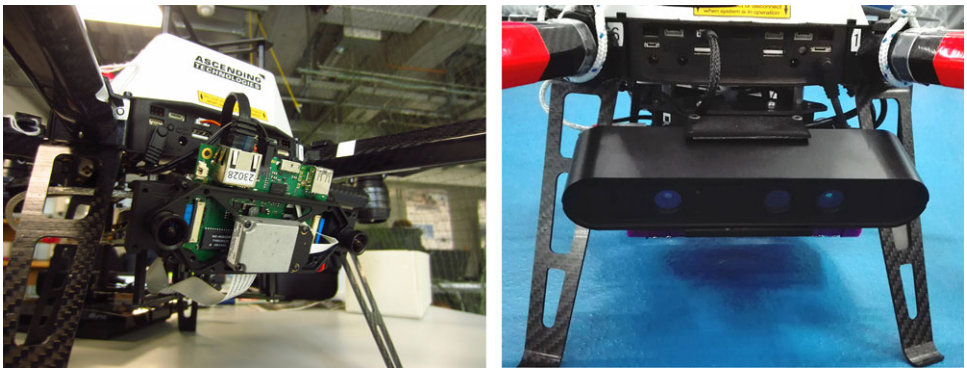


**FIGURE 18** The two main sensors tested on-board the UAV: VI-Sensor (left) and Orbbec Astra (right)

It is a hexacopter with 9 inches propellers, which can lift up to 2 kg, but the maximum nominal flight time of 20 min (without payload) will be reduced accordingly. This is a platform developed and optimized within the framework of different European research projects.

The main sensor on-board the platform was Skybotix's visual-inertial (VI-) sensor, a stereo camera with a calibrated and synchronized IMU. We have modified the payload and the original mount to accommodate different sensors. Our architecture has been validated not only with the VI-sensor but also using an RGB-D camera as the main sensor, in particular the Orbbec Astra, as it can be seen in Figure 18. The sensor is mounted facing forward in the direction of the flight to maximize the information gathered regarding possible nearby obstacles. The orientation of the sensor is customizable as we have manufactured different mounts using a 3D printer. Depending on the main visual sensor mounted on-board the UAV, the module *Sensor-based Odometry* needs to be adapted accordingly to work with the type of data that each sensor delivers.

The on-board embedded computer (Intel NUC with Core i7) is in charge of acquiring sensor data, estimating the aerial robot current state, and generating the appropriate trajectories according to a given navigation goal.

## 5.3 | Controlled tests

To succeed in achieving robust autonomous navigation, the aerial robot must demonstrate both reliability and good performance. Extensive field testing has taken place at the indoor test bed of the Center for Advanced Aerospace Technologies (CATEC). This facility is used to develop and test a wide range of technologies related to autonomous systems. There is a useful volume of 15 × 15 × 5 m; part of it can be seen in Figure 19. The test bed houses an indoor localization system based on 20 VICON cameras, which only needs the installation of passive markers on the objects to locate and/or track. This system is able to provide the position and attitude of each object in real-time with millimetric precision. This tremendously helped us to benchmark the developed algorithms against a very precise ground truth for robot localization, motion planning, or obstacle avoidance with safe distance assurance.

The goal of the controlled tests performed at this indoor test bed was to monitor our development progress and measure flying skills



**FIGURE 19** The indoor test bed at CATEC showing one of the scenarios in which the UAV has been tested, in this case a piping industrial environment

of the aerial robot. We also used the tests to evaluate the effects of changes in hardware and software on the UAV. The flying skills we have systematically tested to assess our progress were the following:

- Ability of the UAV to autonomously takeoff and perform stable hovering.
- Ability of the UAV to follow a pre-planned obstacle-free trajectory.
- Ability of the UAV to generate an obstacle-free trajectory to safely reach a specific waypoint.
- Ability of the UAV to dynamically modify a generated trajectory toward a waypoint to avoid perceived obstacles along the way.

As it is apparent in Figure 19, the test bed structure allows us to secure the UAV from the top via a safety rope to prevent an undesired flight termination. This has been very useful during the development process, as testing new features in the algorithms may result in unexpected behaviors.

The UAV running the proposed architecture has been extensively tested to fulfill all these flying skills, which were meant to guide our development toward high-level autonomous tasks. These tasks are listed below:

- **State estimation**: The purpose of this task is to assess the drift of the visual localization system and state estimator. Collision-free random

15564967, 2018, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/rob.21757 by Indian Institute Of Technology, Wiley Online Library on [28/09/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

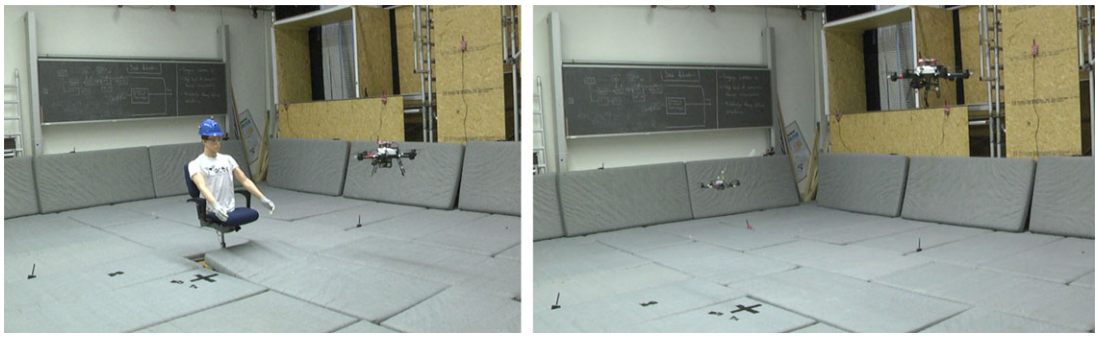
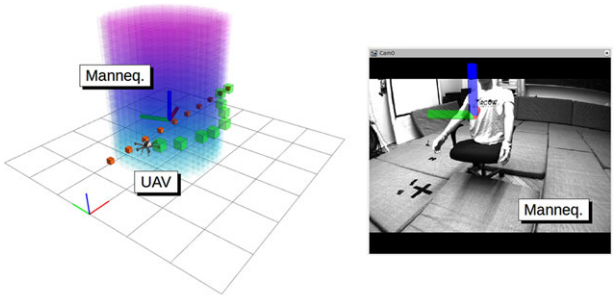
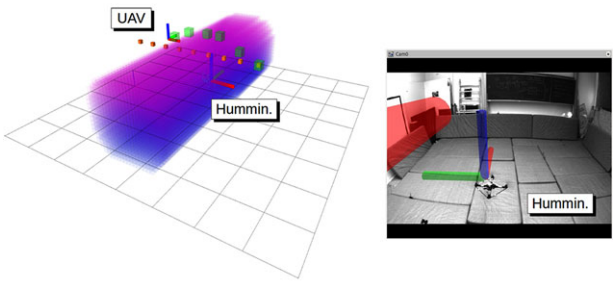


**FIGURE 20** The UAV performing obstacle detection and avoidance of a human worker (left) and another smaller UAV (right)




(a) Mannequin experiment




(b) Multi-UAV experiment

**FIGURE 21** Obstacle detection and trajectory replanning in obstacle avoidance tests. Left images show the occupation matrix around the estimated position of the obstacle and the replanned trajectory (green cubes) to avoid it compared to the original trajectory (orange cubes). Right pictures show the on-board camera image and the estimated relative position of the detected obstacle as a reference frame

trajectories are sent to the UAV and have to be followed as accurately as possible.

- **Reactive obstacle avoidance**: This task benchmarks reactive collision avoidance. Velocity commands are sent to the aerial robot, steering it toward multiple obstacles of increasing difficulty, for example, walls, small sticking objects, or hanging cables. The task is evaluated according to the minimum distance between the UAV and the obstacles during the task execution.
- **Waypoint navigation**: This task benchmarks local map creation as well as planning. First, the arena is explored by following a collision-free trajectory (as in Task 2) that covers the arena quite well, but may leave some unknown spots. After this trajectory is finished, collision-free paths have to be planned to several waypoints with increasing difficulty, and followed by the UAV.

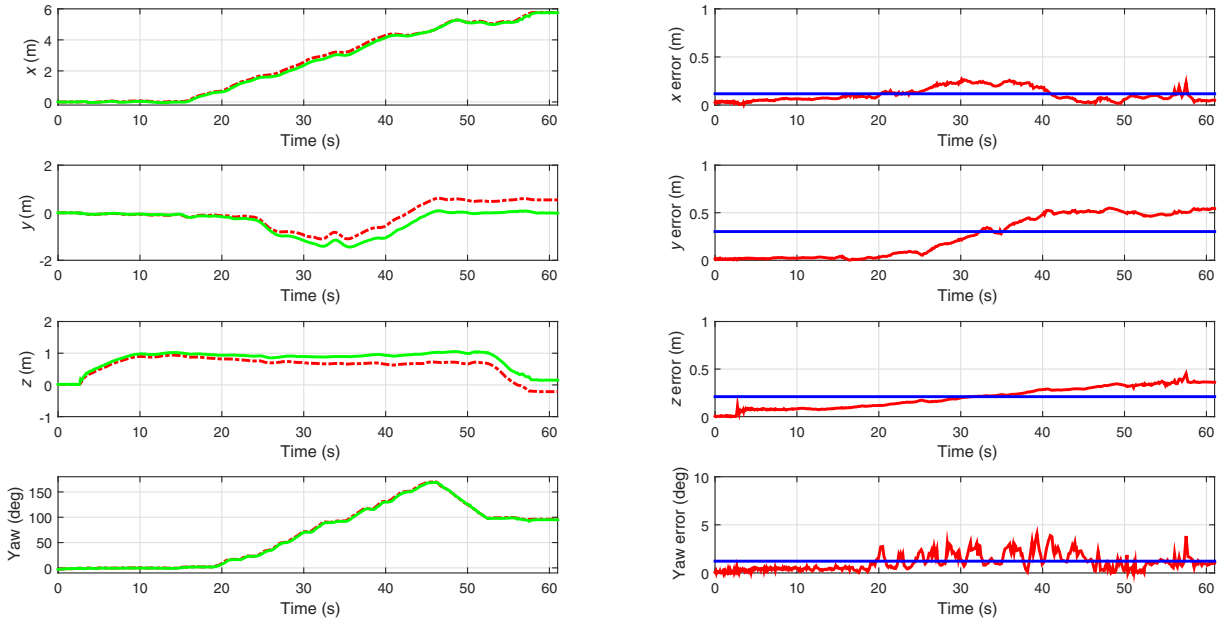
The *Waypoint Navigation* involves the interaction of all the key modules from our architecture: visual odometry, robot localization, global planner, local planner and trajectory tracker. Our core idea is aimed at achieving a robust system to enable the safe integration of UAVs in scenarios where they are currently not able to operate. For this reason, since the only source of information for obstacle avoidance was the frontal vision-based sensor, the concept of flying in any direction was discarded from the beginning. Our system would always turn its front sensor toward the direction of the flight to ensure a safe trajectory and avoid any obstacle that it would encounter. That may result in a slightly slower system; however, this approach is far more robust than moving in any direction without guarantee of flying through an obstacle-free trajectory.

## 5.4 | Obstacle avoidance tests

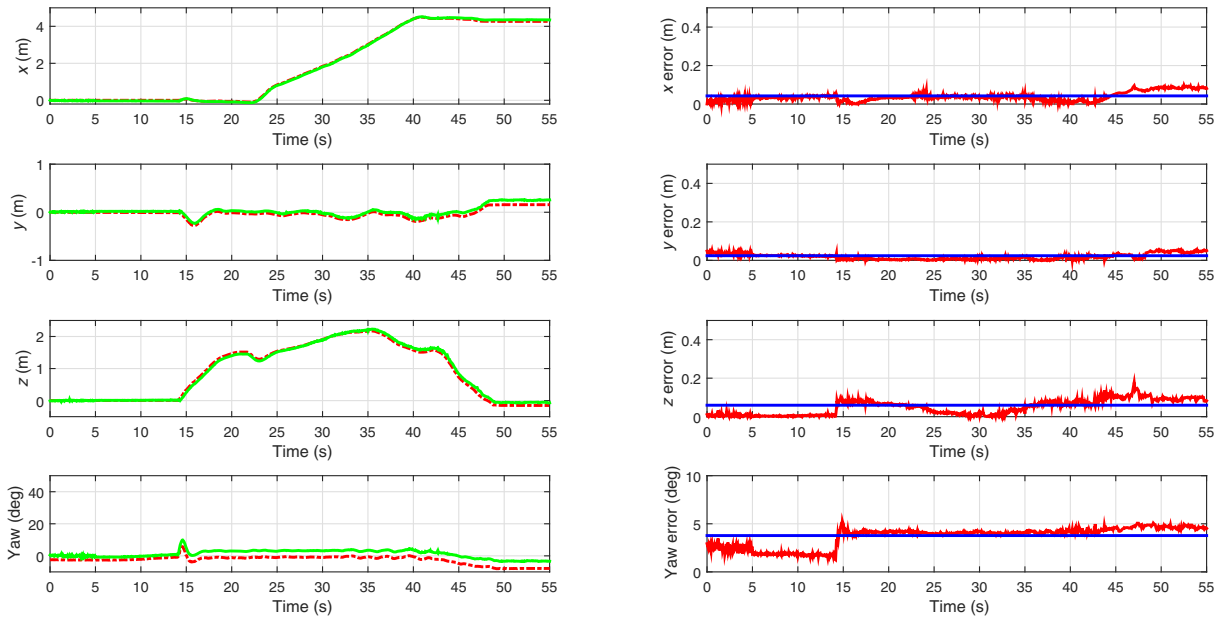
Another set of flight tests was performed using our architecture to validate specific functionalities according to the safe introduction of aerial robots in aircraft manufacturing plants. These tests included tasks related to the interaction of the aerial robot with other agents in its environment, such as human workers or other aerial robots. Hence, obstacle detection and avoidance were basic tasks that our system needed to tackle. Figure 20 shows two of the experiments we carried out. In these experiments, the VI-Sensor was the main visual sensor on-board the UAV. *Visual odometry* was performed using the approach from Ref. 38 based on a stereo camera, which implements a similar pipeline as the one described in Section 2. However, at this time the *robot localization* module did not use the approach from Section 3; instead, we made use of S-PTAM,[54] a stereo SLAM system able to compute the camera trajectory in real-time. It heavily exploits the parallel nature of the SLAM problem, separating the time-constrained pose estimation from less pressing matters such as map building and refinement tasks.

The first experiment aimed at performing autonomous robot navigation while keeping a safe distance from human workers in the factory, emulated by a mannequin. A waypoint was provided to the global planner to send the UAV 5 m forward with a speed of 0.3m/s, enforcing a collision situation where the mannequin is an obstacle in the trajectory commanded to the UAV. Whenever the system detected the mannequin, approximately at a distance of 2m, the local planner automatically took over and recalculated the trajectory to always keep

15564967, 2018, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/rob.21757 by Indian Institute Of Technology, Wiley Online Library on [28/09/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License




(a) Mannequin experiment




(b) Multi-UAV experiment

**FIGURE 22** Estimated UAV position and orientation in obstacle avoidance tests. Left plots show the comparison between the UAV ground-truth captured by the motion capture system (red line) and the estimated UAV localization (green line). Right plots show the computed error associated to each axis (red line) and the RMS error along the trajectory (blue line)

a safe distance from the obstacle and also changing the *yaw* angle so it remained at sight while it was being avoided, until the UAV returned to the original trajectory generated from the global planner. The top part of Figure 21 shows the human worker detection and the replanned trajectory around the inflated occupancy matrix centered on the estimated obstacle position. Our target safe distance to maintain from the human worker was 1.25m. During the experiment, we achieved an average separation distance from the UAV to the mannequin of 1.35m. Figure 22 shows the UAV pose estimation results from the *robot localization* module. Changes in yaw slightly affected the localization, especially in the *y* axis, but errors in general are acceptable, as Table 1 shows.

**TABLE 1** RMS localization errors

| | *x* (m) | *y* (m) | *z* (m) | *yaw* (°) |
|---|---|---|---|---|
| Mannequin experiment | 0.117 | 0.302 | 0.209 | 1.226 |
| Multi-UAV experiment | 0.042 | 0.024 | 0.059 | 3.776 |

15564967, 2018, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/rob.21757 by Indian Institute Of Technology, Wiley Online Library on [28/09/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

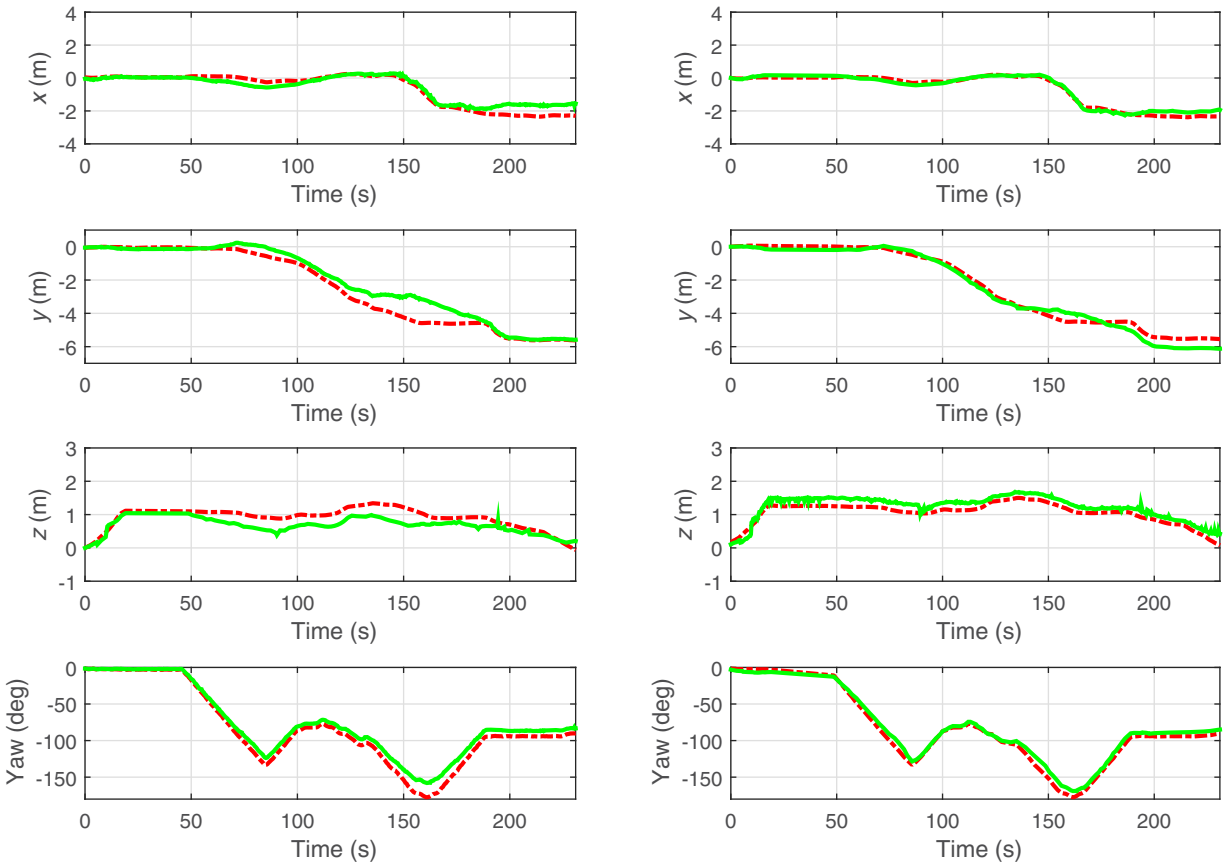



**FIGURE 23** Estimated UAV position and orientation in the validation test. The plots show the UAV ground-truth position and yaw captured by the motion capture system (red line), and the estimated UAV position and orientation (green line) using the proposed approaches for *visual odometry* (left plots) and *robot localization* (right plots)

The second experiment involved autonomous conflict avoidance between aerial robots sharing the same space in large factories. A second smaller UAV (AscTec Hummingbird) was used and commanded to perform an off-line pre-planned trajectory, navigating using the motion capture system of the flying arena. Again, the global planner was sent a waypoint so that the UAV needed to move 5m forward at 0.3m/s, and hence the generated trajectory implied a conflict with the one commanded to the smaller UAV, which was sent a perpendicular trajectory. The system detected the obstacle thanks to the visual sensor at around 1.5m, and then the local planner automatically resolved the conflict by increasing the flight height as it is shown in Figure 21. The safe distance to the obstacle was again 1.25m, but the replanning strategy was different due to the height change. During the experiment we achieved an average separation distance (on the *XZ* plane) of 1.32m (with a maximum peak of 1.54m and a minimum peak of 1.14 m), very close to the target safe distance. Figure 22 shows the UAV pose estimation results, and as it can be observed also in Table 1, errors are much lower in this case, especially because the trajectory did not involve considerable changes in the *yaw* angle.

## 5.5 | Validation test

A specific experimental setup has been conceived to demonstrate the presented approach and explicitly show detailed information about the underlying algorithms. This time, the UAV main visual sensor was the Orbbec Astra (RGB-D camera), and the modules *sensor-based odometry* and *robot localization* were adapted accordingly.

The aerial robot in Figure 17 performed a flight based on local sensing for localization and was sent a navigation waypoint in a specific position of its environment. A 3D map of the scenario is delivered to the global planner to calculate a safe trajectory to reach the waypoint, shown in Figure 26(a). Along the trajectory, the UAV encounters an unforeseen obstacle that forces the local planner to recalculate the trajectory to safely reach the originally commanded waypoint. All the results presented in this paper were obtained online. The main objective of the experiment is to demonstrate the suitability of the architecture in real-time during regular operations.

The 3D map of the area was acquired using a RGB-D sensor augmented with several passive markers to retrieve its pose from CATEC's testbed motion capture system, and hence use that localization to obtain an accurate occupancy grid. This map was showed in Section 3 and can be seen in Figure 5.

Regarding the aerial robot localization, the results from the *visual odometry* and the *robot localization* modules are first presented. The estimated position and yaw angle during the experiment can be seen in Figure 23, in which the left plots show the results from the *visual odometry* approach explained in Section 2, and the right plots show the results from the *robot localization* approach explained in Section 3.

15564967, 2018, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/rob.21757 by Indian Institute Of Technology, Wiley Online Library on [28/09/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

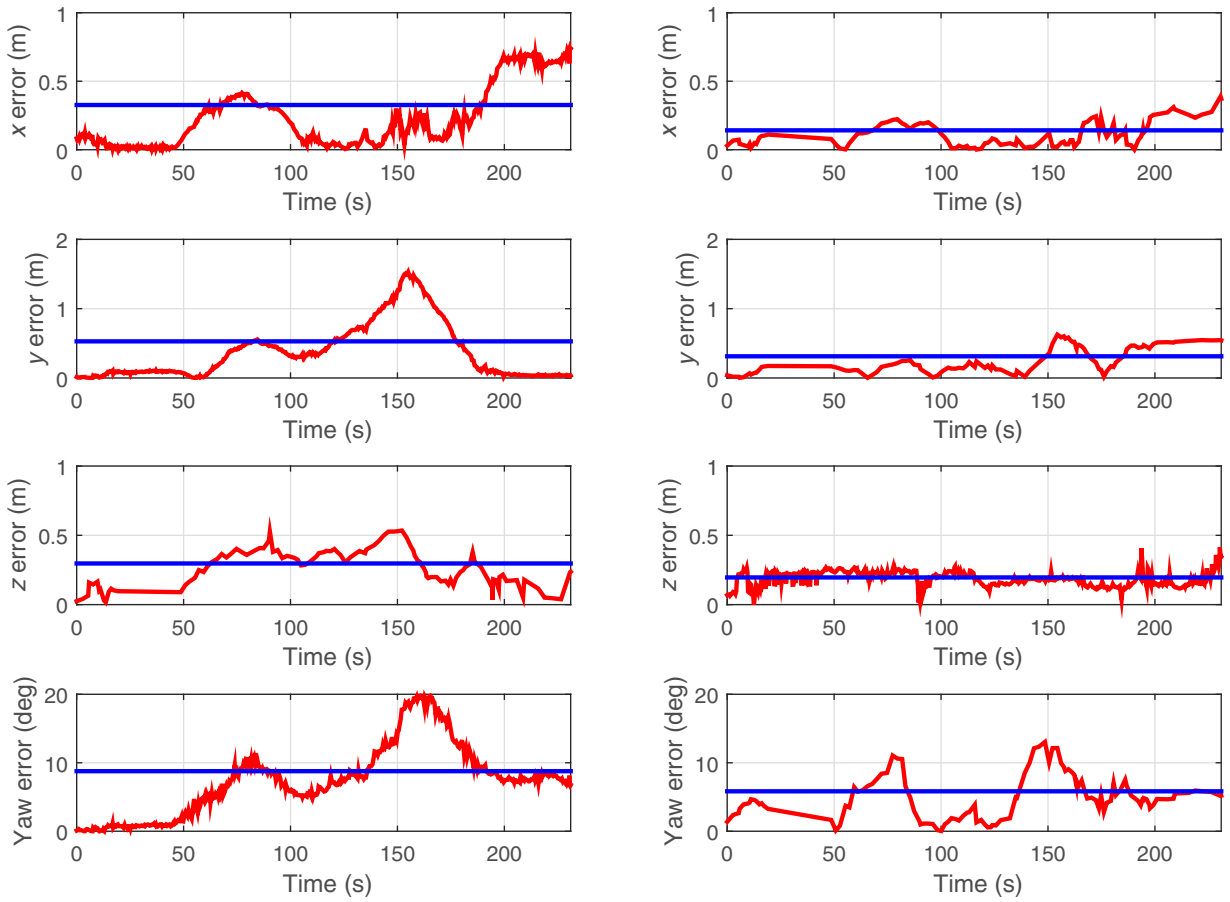



**FIGURE 24** Position and orientation errors through time in the validation test. The plots show the computed error of the estimated position and orientation with respect to the ground-truth (red line), using the proposed approaches for *visual odometry* (left plots) and *robot localization* (right plots). The estimated RMS errors for each axis are also shown (blue line)

**TABLE 2** RMS localization errors

| | *x* (m) | *y* (m) | *z* (m) | *yaw* (°) |
|---|---|---|---|---|
| *Visual odometry* | 0.326 | 0.528 | 0.297 | 8.772 |
| *Robot localization* | 0.141 | 0.313 | 0.196 | 5.833 |

Roll and pitch angles are observable through the use of the on-board IMU and hence are not shown in the figures, since they are directly integrated in both algorithms (odometry and localization). The plots include the ground-truth pose provided by the motion capture system at the indoor test bed.

In both cases, the estimations closely follow the ground-truth during all the experiment; however, it can be noted that the odometry measurements exhibit slightly higher deviations than those from the localization algorithm. Figure 24 shows the absolute value of the errors in the estimations during the trajectory execution. Again the left plots correspond to the *visual odometry*, while the right plots correspond to the *robot localization*. The plot also shows the computed RMS error for each axis, and as expected the errors are significantly smaller in the case of the localization algorithm. In particular, the plots show serious drifts in the visual odometry estimations, especially in *Y* and *yaw*, which the particle filter for robot localization was able to decrease o readjust the UAV-estimated pose. Table 2 shows the RMS errors for each component (*X*, *Y*, *Z* and *yaw*) to quantitatively assess this improvement in the aerial robot localization.

The total RMS error in position is approximately 0.23 m, which can be considered acceptable for robot navigation. Errors in *Y* are slightly higher than the rest of estimations due to the longer distance traveled through this axis. Besides, if the scenario does not show enough visual features or objects within the working range of the sensor, the odometry system suffers especially when sudden changes take place in yaw angle, which happened consistently during this experiment to avoid the unexpected obstacle.

The algorithms run on an i7 Linux laptop with 2 Cores (4 threads). The particle filter for robot localization is configured work with 500 hypotheses. The visual odometer took an average 90% of a single thread and the proposed MCL algorithm the 55% of a single thread. This configuration leaves plenty of computation for robot planning and navigation.

To quantitatively assess the performance of our localization approach, we have tested the data from the validation test against other popular approaches that make use of RGB-D data for robot localization, either visual odometry and mapping, such as CCNY_RGBD*,

* http://wiki.ros.org/ccny_rgbd

15564967, 2018, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/rob.21757 by Indian Institute Of Technology, Wiley Online Library on [28/09/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

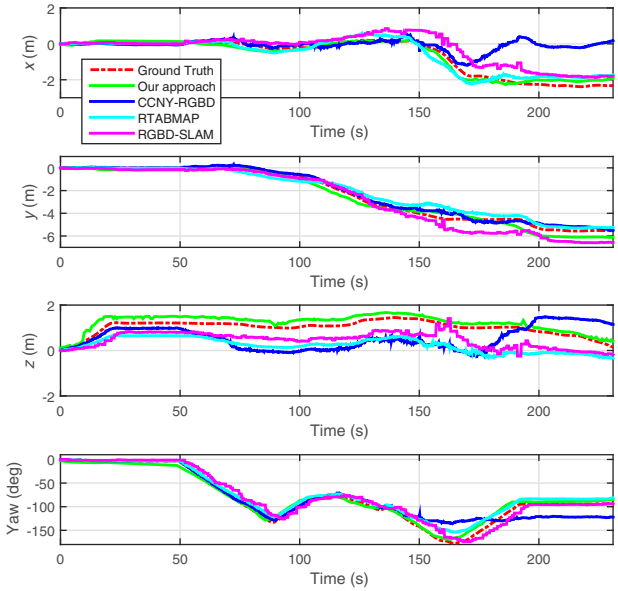



**FIGURE 25** Estimated UAV position and orientation using other approaches based on RGB-D sensors

**TABLE 3** RMS localization errors

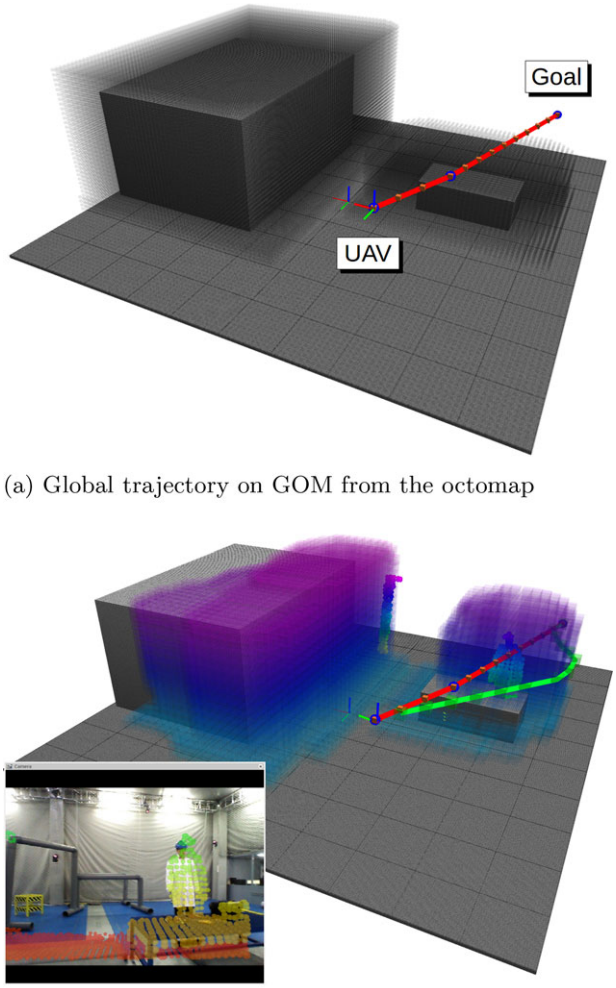
| | *x* (m) | *y* (m) | *z* (m) | *Yaw* (°) |
|---|---|---|---|---|
| Our approach | 0.141 | 0.313 | 0.196 | 5.833 |
| CCNY RGBD tools | 1.075 | 0.220 | 0.710 | 15.917 |
| RTAB map | 0.285 | 0.174 | 0.734 | 9.25 |
| RGBD-SLAM | 0.470 | 0.583 | 0.617 | 9.849 |

or SLAM approaches, such as RTAB-Map* or RGBD-SLAM†. The plots in Figure 25 show the performance of the same flight data processed off-line using these approaches, and Table 3 summarizes general RMS errors compared to those from our approach.

It can be seen how the RMS errors in *X*, *Y*, and *Z* are very similar to the errors of the proposed odometry approach. So our approach provides an accuracy at the level of the state of the art while keeping robustness and computational efficiency as key features. Notice that we did not finetune the parameters of the algorithms, so a better accuracy could be achieved.

From the point of view of the global planner, the experiment consists in reaching a specific goal position, that is at the right and behind the UAV from its initial location, by generating a path as short as possible in an admissible time, previously knowing the fixed obstacles (e.g., a workbench) in the global map showed in Figure 26(a). Therefore, the global planner is configured to explore symmetrically in the three *XYZ* dimensions: $z_w = 1$, but focusing the searchono the goal position: $w = 10$, thereby reducing the planning time. The resulting global trajectory passes over the table, it is very close to the shortest path and it takes only 0.21 seconds to be obtained.

The global trajectory waypoints are computed to reach the yaw in advance, as it was explained in the global planner subsection, so




(a) Global trajectory on GOM from the octomap

(b) First replanned trajectory on LOM from the point cloud shown in the on-board camera

**FIGURE 26** Global and local planners trajectories and occupancy maps during the experiment

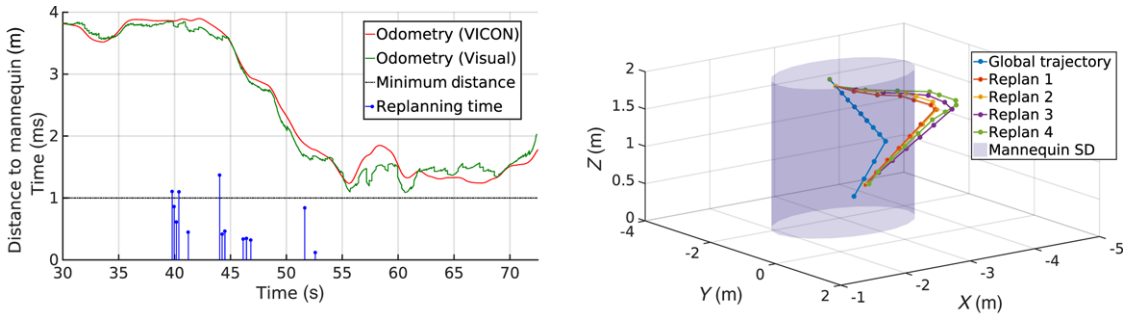
the UAV starts turning around from the current location to the first waypoint direction. As it can be observed in Figure 26(b), the unforeseen obstacle (a worker around the workbench) is perceived before the vehicle starts moving horizontally, so the local planner avoids it in advance. The replanner algorithm is configured to replan as quickly as possible: $w = 10$ and $z_w = 4$, prioritizing the horizontal paths in addition. The resulting replanning trajectories takes a maximum of 1.14 milliseconds to generate a new path to avoid the obstacle, with a path length increment of 1.42 meters with respect to the global trajectory length, being the horizontal inflation of 1 meter and the algorithm resolution of 0.1 meters. Figure 27(a) shows that the UAV always keeps a safe distance even if the human detector is not enabled, which is the case of this experiment.

Actually, a total of 13 consecutive replanning trajectories have been computed since the on-board camera sees the unknown obstacle, but as it is shown in Figure 27, all the trajectories are consistent and very close to each other. This relatively high number of replanning operations in the experiment is due to the fact that new captured points of the obstacle are continuously captured as the UAV moves around

* http://wiki.ros.org/rtabmap

† http://wiki.ros.org/rgbdslam

15564967, 2018, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/rob.21757 by Indian Institute Of Technology, Wiley Online Library on [28/09/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License




(a) Replanning times and distances to unforeseen obstacle

(b) First four consecutive replanned trajectories

**FIGURE 27** Times and resulting trajectories to avoid the unforeseen obstacle



**FIGURE 28** Sample RGB images from the front camera during high-speed tests

avoiding it, which makes the replanned trajectory increasingly safer. The average replanning time is 0.642 milliseconds, even though some replanning computations are very simple problems to solve a trajectory from a waypoint to the next one.

The rates at which the different modules of the architecture are running or carrying out computations are discussed below. In the obstacle avoidance tests, the VI-Sensor is delivering stereo-image pairs at 20 Hz. *Sensor-based Odometry* processes each frame in approximately 50 ms after downsampling the input images, which allows for real-time computation. *robot localization* based on S-PTAM delivers pose estimations at around 15 Hz, thanks to internal optimizations. Since the *controller* is running at 100 Hz, the state estimation frequency is increased by combining it with IMU measurements through a filter. Regarding the *trajectory planner*, the update rate of the auxiliary obstacle map works at lower frequencies because it needs to wait until the points are seen several times to be considered obstacles. The number of times needed to be seen is set to 5, which produces an obstacle in the map after 25 ms of continuous sight. Finally, the *checkReplan* function from Algorithm 2 is performed every 100 ms in order not to overload the processing unit. Regarding the validation test, the main difference is related to the main sensor and the state estimation; the Orbbec Astra was configured to deliver images at 15 Hz (half of its full operational frequency) to leave processing power to the modules of our architecture. *Sensor-based Odometry* is able to generate motion estimations also at 15 Hz, as well as our *robot localization* approach based on MCL, allowing for real-time on-line state estimation.

## 5.6 | High-speed tests

To prove the robustness of our localization approach, also under fast motions of the UAV, a series of additional tests have been performed. For safety reasons, the tests have consisted in manually carrying the UAV to reproduce high-speed flights in the same scenario at CATEC's test bed where the validation test took place. Sample images from these tests are shown in Figure 28, where motion blur is clearly visible.

Two different strategies have been validated. First, ample movements involving displacements mainly in the $y$ axis have been performed. Second, more reduced movements in the $y$ axis, with more changes in *yaw*, have been carried out at faster motions.

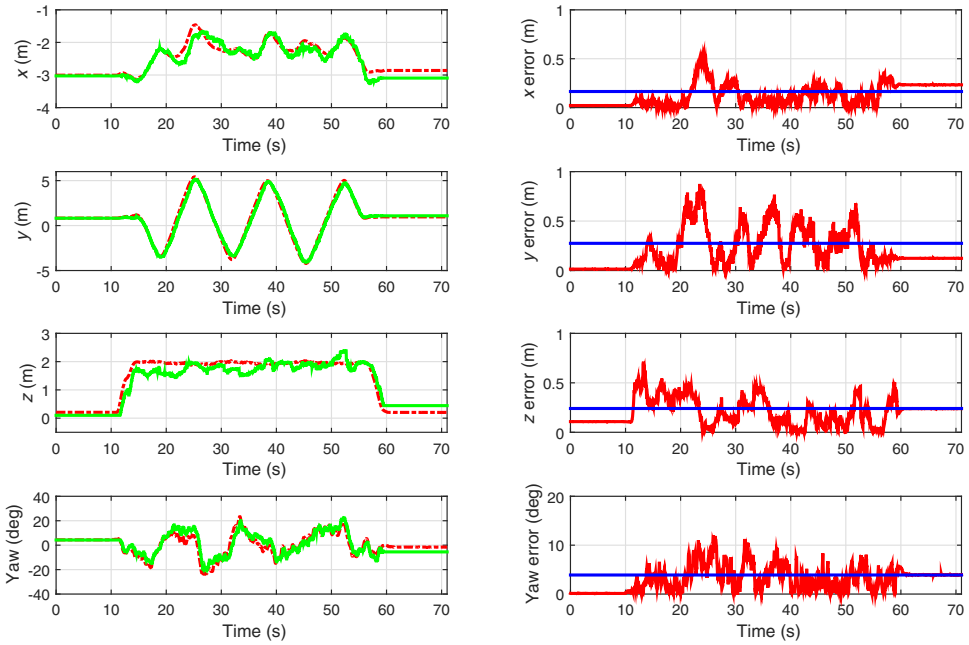
The comparison between ground-truth data and our localization estimation can be seen in Figure 29, as well as error plots. In the first test, the UAV speed reaches 1.2m/s, since it traverses nearly 9 m in around 7 s several times, between $t = 20$s and $t = 55$s. In the second test, higher speeds of up to 1.5m/s are reached especially between $t = 14$s and $t = 20$s.

As it can be observed, RMS errors in position are below 0.3m, whereas errors in *yaw* rarely exceed 10 deg. The estimations closely follow the ground-truth, except for the final part of the second high-speed test, where $z$ estimations do not decrease, whereas the UAV was manually carried to the floor. This issue should not represent any inconvenience in regular UAV operations, since the rest of experimental results from this section did not show similar behaviors.

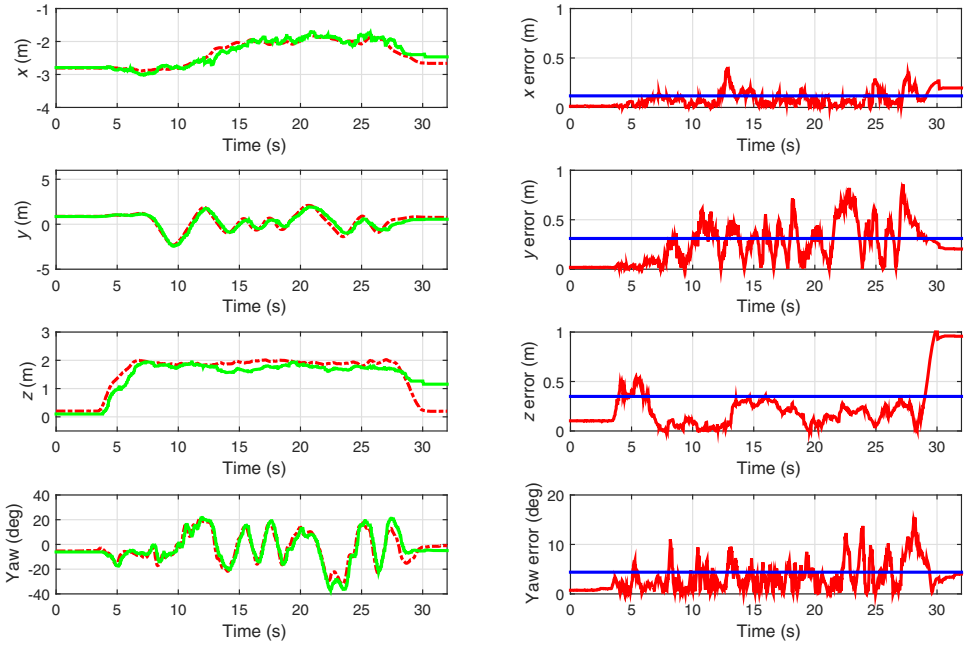
## 6 | LESSONS LEARNED

While the system performs well, there is still room for improvement in its performance and the development process. Field testing is one of the most expensive tasks when it comes to developing an autonomous system because the aerial platform must be fully operational before carrying out flight experiments, a space for testing must be accessible and specific qualified staff must be available (including a safety pilot).

15564967, 2018, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/rob.21757 by Indian Institute Of Technology, Wiley Online Library on [28/09/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License




(a) First high-speed test




(b) Second high-speed test

**FIGURE 29** Estimated UAV position and orientation in high-speed tests. Left plots show the comparison between the UAV ground-truth captured by the motion capture system (red line) and the estimated UAV localization (green line). Right plots show the computed error associated to each axis (red line) and the RMS error along the trajectory (blue line)

Besides, if testing outdoors the weather must behave, and depending on national regulations a valid license for the UAV pilot might be required.

An important factor to take into consideration when testing aerial robots is their flight autonomy. Degradation of batteries is a significant issue that we have encountered during extensive test campaigns, forcing us to test within limited flight times until replacements can be available. Simulation environments such as Gazebo* have been very useful for testing the behavior and integration of different modules, especially those related to motion planning. However, the simulations are still limited to provide accurate information regarding real-world scenarios or behaviors of the actual system.

* http://gazebosim.org/

15564967, 2018, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/rob.21757 by Indian Institute Of Technology, Wiley Online Library on [28/09/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

A very convenient tool that the team has extensively used during all of our testing is the *node_manager* developed by Fraunhofer FKIE.* It is a graphical interface to manage nodes, topics, services, parameters, and launch files in a ROS network. It seamlessly handles the discovery of running ROS masters in the local network and the synchronization of the local ROS master to the discovered remote masters. This is very useful because all the algorithms are running in the on-board computer, and then visualization and debugging tools can be used in the ground computer to watch live sensor data or check the expected behavior of specific algorithms that are under test.

Another useful tool that has been widely used for testing the *trajectory planner* and *trajectory tracker* modules is the *RotorS*†.[55] It is a UAV simulator based on Gazebo that provides some multi-rotor models, simulated sensors such as the VI-Sensor, and different possibilities for the simulated UAV controllers.

Most of the environments in which our system has been tested have been static scenarios. While significant progress has been made during field testing, aerial robots must be able to perceive and interact with dynamic environments to be introduced in real-world scenarios. Moreover, our system can only handle limited types of obstacles, such as human workers or other UAVs, to carry out a specific set of actions accordingly.

The ability to evaluate the performance of the aerial robot in an experiment is a key factor in field testing. Sometimes a tight time schedule has forced us to carry out extensive test campaigns, during which it was not unusual to adjust several parameters from the localization and planning approaches *on the fly*. This leaves room for subjectivity, since human errors can easily arise. It should be a good idea to develop automated methods to quantitatively assess recorded data to evaluate the robot performance. This will help gather statistical evidence from the behavior of the different algorithms integrated in the architecture, allowing for proper evaluation.

## 7 | CONCLUSIONS

This article presents the overall software architecture and field experience gathered after extensive testing of a UAV platform for autonomous navigation in GPS-denied areas, demonstrating the robustness of the approach in different situations.

Besides, the article gathers specific innovations in the different modules of the proposed architecture to achieve safe and reliable autonomous navigation of the aerial robot. An implementation of MCL suitable for operation in 3D environments has been presented. Visual odometry based on an RGB-D sensor is used along with a previously known map of the environment to provide reliable and accurate four-dimensional (*x*, *y*, *z*, and *yaw*) estimation. Regarding motion planning, an innovative variant of the Lazy Theta* algorithm has been developed and tested, to decrease planning times, get near-shortest paths, and deal with actual visual sensor restrictions. The use of the proposed motion planners based upon this algorithm is capable of enabling the introduction of UAVs into environments that are not necessarily fully known in advance.

The authors are currently preparing to release the source code of some of the algorithms, and add the corresponding information in the ROS wiki to be publicly available. Although the presented work is based on specific implementations of the different modules in the architecture, special focus has been put in maintaining standard and well-defined interfaces between all of them, to be reusable and extensible.

Developing this architecture for an autonomous robot has allowed us to test a platform that would perform in real situations and solve significant research problems. Future work will focus on taking into account that such systems would perform autonomous navigation in a wide variety of scenarios, not only the ones that have been considered so far and which in some cases might represent simplified versions of real world environments.

## ACKNOWLEDGMENTS

This research is a result of a partnership between the University of Seville and the Center for Advanced Aerospace Technologies (CATEC). Thanks also goes to Angel Luis Petrus.

This work has been partially supported by the EuRoC project funded by the EU FP7 under grant agreement CP-IP 608849, AEROARMS project funded by the EU H2020 under grant agreement No: 644271, AEROMAIN project (DPI2014-5983-C2-1-R) and OCELLIMAV project funded by the Spanish Government, grant TEC2014-61708-EXP.

## ORCID

*Francisco J. Perez-Grau* http://orcid.org/0000-0003-3372-7348

## REFERENCES

1. Lim H, Park J, Lee D, Kim HJ. Build your own quadrotor: open-source projects on unmanned aerial vehicles. *IEEE Robot Autom Mag*. 2012;19(3):33–45.
2. Oleynikova H, Burri M, Lynen S, Siegwart R. Real-time visual-inertial localization for aerial and ground robots. In: *2015 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2015:3079–3085.
3. Burri M, Oleynikova H, Achtelik MW, Siegwart R. Real-time visual-inertial mapping, re-localization and planning onboard MAVS in unknown environments. In: *2015 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. Hamburg, NJ: IEEE; 2015:1872–1878.
4. Schmid K, Tomic T, Ruess F, Hirschmller H, Suppa M. Stereo vision based indoor/outdoor navigation for flying robots. In: *2013 IEEE/RSJ International Conference on Intelligent Robots and Systems*. Tokyo, 2013:3955–3962.
5. Michael N, Shen S, Mohta K, et al. Collaborative mapping of an earthquake-damaged building via ground and aerial robots. *J Field Robot*. 2012;29(5):832–841.
6. Heng L, Honegger D, Lee GH, et al. Autonomous visual mapping and exploration with a micro aerial vehicle. *J Field Robot*. 2014;31(4):654–675.

* http://wiki.ros.org/node_manager_fkie

† http://wiki.ros.org/rotors_simulator

15564967, 2018, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/rob.21757 by Indian Institute Of Technology, Wiley Online Library on [28/09/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

7. Davison A, Reid I, Molton N, Stasse O. MonoSLAM: real-time single camera SLAM. *IEEE Trans Pattern Anal Mach Intell*. 2007;29(6):1052–1067.
8. Pinies P, Lupton T, Sukkarieh S, Tardos J. Inertial Aiding of Inverse Depth SLAM using a Monocular Camera. In: *Proceedings 2007 IEEE International Conference on Robotics and Automation*, Roma, 2007:2797–2802.
9. Achtelik MW, Lynen S, Weiss S, Kneip L, Chli M, Siegwart R. Visual-inertial SLAM for a small helicopter in large outdoor environments. In: *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*. Vilamoura, 2012.:2651–2652
10. Weiss S, Scaramuzza D, Siegwart R. Monocular-SLAM based navigation for autonomous micro helicopters in GPS-denied environments. *J Field Robot*. 2011;28(6):854–874.
11. Bloesch M, Omari S, Hutter M, Siegwart R. Robust visual inertial odometry using a direct ekf-based approach. In: *2015 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS). 2015* Piscataway, NJ: IEEE; 2015:298–304.
12. Honegger D, Meier L, Tanskanen P, Pollefeys M. An open source and open hardware embedded metric optical flow CMOS camera for indoor and outdoor applications. In: *2013 IEEE International Conference on Robotics and Automation*. Karlsruhe, 2013:1736–1741.
13. Mebarki R, Lippiello V, Siciliano B. Vision-based and IMU-aided scale factor-free linear velocity estimator. *Autonom Robot*, 2016;41(4):903–917.
14. Kneip L, Chli M, Siegwart R. Robust real-time visual odometry with a single camera and an IMU. In: Jesse Hoey, Stephen McKenna, Emanuele Trucco, eds. *Proceedings of the British Machine Vision Conference*, September 2011. Durham, UK: BMVA Press; 2011:16.1–16.11.
15. Geiger A, Ziegler J, Stiller C. StereoScan: dense 3D reconstruction in real-time. In: *2011 IEEE Intelligent Vehicles Symposium (IV)*. Baden-Baden, IEEE; 2011:963–968.
16. Endres F, Hess J, Engelhard N, Sturm J, Cremers D, Burgard W. An evaluation of the RGB-D SLAM system. In: *2012 IEEE International Conference on Robotics and Automation (ICRA)*. Saint Paul, MN; 2012:1691–1696.
17. Kerl C, Sturm J, Cremers D. Robust odometry estimation for RGB-D cameras. In: *2013 IEEE International Conference on Robotics and Automation (ICRA)*. Karlsruhe, IEEE; 2013:3748–3754.
18. Winterhalter W, Fleckenstein F, Steder B, Spinello L, Burgard W. Accurate indoor localization for RGB-D smartphones and tablets given 2D floor plans. In: *2015 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. Hamburg, 2015:3138–3143.
19. Mur-Artal R, Montiel JMM, Tardos JD. ORB-SLAM: a versatile and accurate monocular SLAM system. *IEEE Trans Robot*. 2015;31(5):1147–1163.
20. Dryanovski I, Valenti RG, Xiao J. Fast visual odometry and mapping from RGB-D data. In: *2013 IEEE International Conference on Robotics and Automation*. Karlsruhe, 2013:2305–2310.
21. Labbé M, Michaud F. Online global loop closure detection for large-scale multi-session graph-based SLAM. In: *2014 IEEE/RSJ International Conference on Intelligent Robots and Systems*. Chicago, IL: 2014:2661–2666.
22. Kaess M, Ranganathan A, Dellaert F. ISAM: Incremental smoothing and mapping. *IEEE Trans Robot*. 2008;24(6):1365–1378.
23. LaValle SM. Rapidly-exploring random trees: A new tool for path planning. *Tech Rep*. 1998. Accessed September 2017. http://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.35.1853
24. LaValle SM, Kuffner JJ Jr. Rapidly-exploring random trees: progress and prospects, *Algorithmic and Computational Robotics: New Directions*. 2000. Accessed September 2017. http://citeseer.ist.psu.edu/viewdoc/summary?doi=10.1.1.36.7457
25. Bohlin R, Kavraki LE. Path planning using lazy PRM. In: *Proceedings 2000 ICRA. Millennium Conference. IEEE International Conference on Robotics and Automation*, San Francisco, CA, April 24–28, 2000. Vol. 1, IEEE; 2000:521–528.
26. Karaman S, Frazzoli E. Sampling-based algorithms for optimal motion planning. *Int J Robot Res*. 2011;30(7):846–894.
27. Kuffner JJ, LaValle SM. RRT-connect: an efficient approach to single-query path planning. In: *Robotics and Automation, 2000. Proceedings 2000 ICRA'. Millennium Conference. IEEE International Conference on Robotics and Automation*, San Francisco, CA, April 24–28, 2000. Vol. 2. IEEE; 2000:995–1001.
28. Oleynikova H, Burri M, Taylor Z, Nieto J, Siegwart R, Galceran E. Continuous-time trajectory optimization for online UAV replanning. In: *2016 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. Daejeon, NJ: IEEE; 2016:5332–5339.
29. Sucan IA, Chitta S. Moveit! http://moveit.ros.org, 2013. Accessed September 2017.
30. Hart PE, Nilsson NJ, Raphael B. A formal basis for the heuristic determination of minimum cost paths. *IEEE Trans Syst Sci Cybernet*. 1968;4(2):100–107.
31. Nash A, Daniel K, Koenig S, Felner A. Theta^*: any-angle path planning on grids. In: *Proceedings of the National Conference on Artificial Intelligence*. Vol. 22. Palo Alto, CA: AAAI Press; 2007:1177.
32. Carsten J, Ferguson D, Stentz A. 3D Field D: improved path planning and replanning in three dimensions. In: *2006 IEEE/RSJ International Conference on Intelligent Robots and Systems*. Beijing, NJ: IEEE; 2006:3381–3386.
33. Nash A, Koenig S, Tovey C. Lazy Theta*: Any-angle path planning and path length analysis in 3D. In: *Proceedings of the Twenty-Fourth AAAI Conference on Artificial Intelligence (AAAI'10)*. AAAI Press; 147–154.
34. Pivtoraiko M, Mellinger D, Kumar V. Incremental micro-UAV motion replanning for exploring unknown environments. In: *2013 IEEE International Conference on Robotics and Automation (ICRA)*. Piscatway, NJ: IEEE; 2013:2452–2458.
35. Roussillon C, Gonzalez A, Solà J, et al. *RT-SLAM: A Generic and Real-Time Visual SLAM Implementation*. Berlin, Germany: Springer; 2011:31–40.
36. Kitt B, Geiger A, Lategahn H. Visual odometry based on stereo image sequences with RANSAC-based outlier rejection scheme. In: *2010 IEEE Intelligent Vehicles Symposium (IV)*. San Diego, CA; 2010:486–492.
37. Huang AS, Bachrach A, Henry P, Krainin M, Fox D, Roy N. Visual odometry and mapping for autonomous flight using an rgb-d camera. In: *Proceedings of the International Symposium of Robotic Research*, 2011;100:235–252.
38. Pérez-Grau FJ, Fabresse FR, Caballero F, Viguria A, Ollero A. Long-term aerial robot localization based on visual odometry and radio-based ranging. In: *2016 International Conference on Unmanned Aircraft Systems (ICUAS)*. Arlington, VA: 2016:608–614.
39. Rosten E, Drummond T. Machine learning for high-speed corner detection. In: *Proceedings of the 9th European Conference on Computer Vision (ECCV'06)*. Vol. Part I. Springer: Berlin. 2006:430–443.
40. Rublee E, Rabaud V, Konolige K, Bradski G. ORB: An efficient alternative to SIFT or SURF. In: *2011 International Conference on Computer Vision*. Barcelona, 2011:2564–2571.
41. Lepetit V, Moreno-Noguer F, Fua P. EPnP: an accurate O(n) solution to the PnP problem. *Int J Comput Vis*. 2008;81(2):155.
42. Lourakis MA, Argyros A. SBA: A software package for generic sparse bundle adjustment. *ACM Trans Math Softw*. 2009;36(1):1–30.


15564967, 2018, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/rob.21757 by Indian Institute Of Technology, Wiley Online Library on [28/09/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

43. Thrun S, Fox D, Burgard W, Dellaert F. Robust Monte Carlo localization for mobile robots. *Artif Intell*. 2001;128(1–2):99–141.
44. Pomerleau F, Colas F, Siegwart R, Magnenat S. Comparing ICP variants on real-world data sets. *Auton Robots*. 2013;34(3):133–148.
45. Hornung A, Wurm KM, Bennewitz M. Humanoid robot localization in complex indoor environments. In: *2010 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), Taipei, Taiwan*. Piscatway, NJ: IEEE; 2010:1690–1695.
46. Hornung A, Wurm KM, Bennewitz M, Stachniss C, Burgard W. OctoMap: An efficient probabilistic 3D mapping framework based on octrees. *Auton Robots*. 2013;34(3):189–206.
47. Lau B, Sprunk C, Burgard W. Efficient grid-based spatial representations for robot navigation in dynamic environments. *Robot Auton Syst*. 2013;61(10):1116–1130.
48. Thrun S, Burgard W, Fox D. *Probabilistic Robotics (Intelligent Robotics and Autonomous Agents)*. Cambridge, MA: MIT Press; 2005.
49. Daniel K, Nash A, Koenig S, Felner A. Theta*: any-angle path planning on grids. *J Artif Intell Res*. 2010;39:533–579.
50. Bresenham JE. Algorithm for computer control of a digital plotter. *IBM Syst J*. 1965;4(1):25–30.
51. Maraj CS, Lackey SJ, Badillo-Urquiola KA, Ortiz EC, Hudson IL. Modeling proxemic cues for simulation-based training in virtual environments. In: *Proceedings from MODSIM World Conference, Virginia Beach, VA*. 2015.
52. Niermeyer P, Akkinapalli VS, Pak M, Holzapfel F, Lohmann B. Geometric path following control for multirotor vehicles using nonlinear model predictive control and 3d spline paths. In: *2016 International Conference on Unmanned Aircraft Systems (ICUAS)*. Arlington, VA: IEEE; 2016:126–134.
53. Quigley M, Conley K, Gerkey B, et al. ROS: an open-source robot operating system. In: *ICRA Workshop on Open Source Software*, IEEE International Conference on Robotics and Automation, Kobe, 2009. Vol. 3. 2009:5.
54. Pire T, Fischer T, Civera J, De Cristóforis P, Berlles JJ. Stereo parallel tracking and mapping for robot localization. In: *Intelligent Robots and Systems (IROS), 2015 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. Hamburg, IEEE; 2015:1373–1378.
55. Furrer F, Burri M, Achtelik M, Siegwart R. Robot Operating System (ROS). In: Koubaa A. (Ed.). *Robot Operating System (ROS): The Complete Reference (Volume 1)*. Studies in Computational Intelligence, Vol. 625 Berlin: Springer; 2016: chap. 23.

**How to cite this article:** Perez-Grau FJ, Ragel R, Caballero F, Viguria A, Ollero A. An architecture for robust UAV navigation in GPS-denied areas. *J Field Robotics*. 2018;35:121–145. https://doi.org/10.1002/rob.21757

15564967, 2018, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/rob.21757 by Indian Institute Of Technology, Wiley Online Library on [28/09/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License